

Enhancing Steam Game Recommendations with Network-Derived Features

Bachelor Project – May 2026

IT University of Copenhagen

Aniela Marta Ciecierska

anci@itu.dk

Jakub Piotr Gasior

jaga@itu.dk

Supervised by:

Michele Coscia

mcos@itu.dk

https://github.com/kubaa1501/bachelor_project

Abstract

Building a recommendation model that accurately captures users' preferences and provides personalized suggestions is challenging, and its success depends heavily on the quality and richness of the available features. While traditional non-network features provide useful descriptive information, features derived from the underlying network structure of a platform may capture additional relational signals that are otherwise absent.

This thesis investigates whether network-derived features can improve the performance of machine learning models for recommending *Steam* games. Using a dataset of *Steam* users and games, we trained several models under two conditions: one that used only non-network features and another that used the same features with the addition of network-derived information. The results show that network features significantly improve the performance of all traditional models tested. For example, the *XGBoost Baseline* achieved an *MRR* of 0.8187 ± 0.0006 , whereas *XGBoost* with network features reached 0.9953 ± 0.0001 . Similarly, *Logistic Regression* improved from 0.3384 ± 0.0000 to 0.8437 ± 0.0006 on *MRR* score when network features were included. In contrast, the *Neural Network* and *Graph Convolutional Network* models did not benefit significantly from the additional features, likely due to their overall weaker baseline performance. Further analysis revealed that incorporating network features not only altered the distribution of recommended games but also increased the models' confidence in their predictions.

1 Introduction

Recommendation systems are widely used across major digital platforms. They suggest products on *Amazon*, recommend new friends on *Facebook*, create playlists on *Spotify*, and propose movies on *Netflix*. *Steam* also relies on such systems to help users discover games they may enjoy. Recommendation systems are implemented so commonly because these platforms contain thousands of options to choose from. Without recommendations, users would struggle to discover content that matches their interests. By providing users with personalized suggestions, platforms not only make the experience more enjoyable, but also help users navigate their extensive catalogs more easily. As a result, recommender systems benefit both users and platforms. Users can discover satisfying content more efficiently and spend less time searching, while platforms benefit by attracting more customers, increasing user engagement, and encouraging users to interact with a wider range of content.

For gaming platforms such as *Steam*, providing accurate recommendations is particularly important. Unlike songs or movies, video games are often a serious financial and time investment. Many titles cost significant amounts of money and take tens of hours to complete. As a result, users must be convinced not only that a game matches their preferences, but also that it is worth the commitment. This makes high-quality recommendations especially valuable in the gaming domain.

Building a recommendation model requires capturing a user’s taste and predicting what they are likely to enjoy next. Descriptive features about users and games can provide useful signals: the types of games a user plays, their total playtime, their country, or the genres and popularity of individual games. These non-network features are commonly used in machine learning models for recommendations. However, they primarily describe individual users or games in isolation. They do not capture the relational structure of the platform, how users interact with each other, or how games are connected through shared player communities.

Steam’s social features offer an additional source of information. Users can add each other as friends, forming a social network. Combined with the user-game interactions, this creates a graph structure that represents relationships between users and games. Modeling *Steam* as a graph makes it possible to extract structural information that is not present in non-network features. By computing numerical embeddings that encode this structure, we can capture relationships such as game similarity based on shared player behavior and their social connections. These network-derived features can then be incorporated into traditional machine learning models alongside non-network features. This leads to the central question of this thesis: *To what extent do network-derived features improve the performance of Steam game recommendation models compared to non-network features?*

2 Related Work

Recommendation systems are typically categorized into content-based, collaborative filtering, and hybrid approaches (Ricci *et al.*, 2011). Content-based methods rely on item attributes, such as genres or tags, and recommend items similar to those a user has already interacted with. In contrast, collaborative filtering methods focus on patterns in user-item interactions, recommending items based on the behavior of similar users.

Collaborative filtering approaches, such as those popularized during the *Netflix Prize* (Koren *et al.*, 2009), learn latent representations of users and items. These representations capture hidden patterns in user preferences and item characteristics, enabling similarity estimation even when users have not interacted with the same items directly. A common method in this setting is *Alternating Least Squares (ALS)*, which learns such representations from implicit feedback signals such as playtime (Koren *et al.*, 2009).

In practice, modern recommender systems often combine multiple types of information. In addition to interaction data, they may incorporate item metadata, user activity, and popularity signals. These features can be used within machine learning models, including methods such as *gradient boosting*, which have been successfully applied in recommendation tasks (Sinaga & Wibowo, 2024).

However, these approaches typically ignore the underlying social structure of the platform. On *Steam*, this is particularly relevant: users are connected through a friendship network that

provides relational information beyond individual interactions. *Steam*’s own Interactive Recommender is trained only on playtime histories and does not incorporate tags, reviews, or social network information (Valve Corporation, 2023). While effective, this collaborative filtering approach does not exploit the structure of the friendship graph.

Recent work has therefore explored incorporating network structure into similarity estimation. *Coscia* (2020) introduced a network-aware distance measure that accounts for the topology of the underlying graph by weighting user disagreements according to their structural proximity. This measure enriches simple co-ownership patterns with information about the social network. Building on this idea, prior work has used this distance matrix as the basis for dimensionality reduction, producing network-aware embeddings that capture structural relationships between items in a low-dimensional space (Damstrup, Madsen & Coscia, 2023). Similar techniques have also been applied to cultural domains analogous to video games (Coscia, 2024).

Motivated by these developments, we derived network-aware game embeddings from the *Steam* friendship graph using *Coscia*’s distance measure, followed by *Principal Component Analysis* (PCA) to obtain compact representations. These embeddings served as an additional feature source that captured relational information not present in non-network features. By incorporating these features into traditional machine learning models and comparing them with graph-based neural approaches, we evaluated the extent to which network-derived features improved recommendation performance on *Steam*.

3. Data

3.1 Data Collection

To fully control what information and how much we collect, we decided to scrape our data from *Steam* with an *API key* (Valve Corporation 2026). This allowed us to work with data that contained all available attributes that we found useful to build a recommendation model and have the most recent data from the platform.

To collect the data, we crawled through *Steam*’s friendships network. We started from our supervisor’s *ID*, and we continued sampling the data with a randomized graph crawl. The crawler maintained: a set of visited users, a set of discovered users that were not yet visited, and attributes of the visited users. At each iteration, one discovered user was randomly selected, marked as visited, their attributes stored, and their friends’ *IDs* added to the discovered set. We ended up with:

- **Visited users:** 99,924
- **Discovered users:** 6,084,524

We aimed to sample information about *100,000* users, but due to problems with the *Steam API* during the crawl, some visited users did not have any attributes available and therefore were instantly not taken into consideration.

Those attributes were collected:

- **user attributes:** *steamid, public, country, account_created, friend_steamid, friend_count*
- **user-game attributes:** *playtime_minutes*

To enrich our game attributes, we separately scraped data about the games owned by visited users, which were:

- **game attributes:** *appid, name, genres, developer, publisher, platforms, release_date*

Our approach allowed us to collect a broad sample of the reachable *Steam* friendship graph. By randomly selecting the next user to visit, we avoided the inherent bias of using *Breadth First Search (BFS)*, which would collect the data from the users layer by layer and result in over-representing the community close to the starting point seed. Our method allowed us to sample users many steps from the origin, capturing different regions of the network.

3.2 Data Cleaning

Steam allows users to control their privacy settings (*Valve Corporation 2026*). Accounts can be set to either private, hiding almost all information, or public. During the sampling process, both private and public users were visited. Almost 90% of the users encountered had public accounts, while a bit over 10% had their accounts set to private (see *Figure 1*).

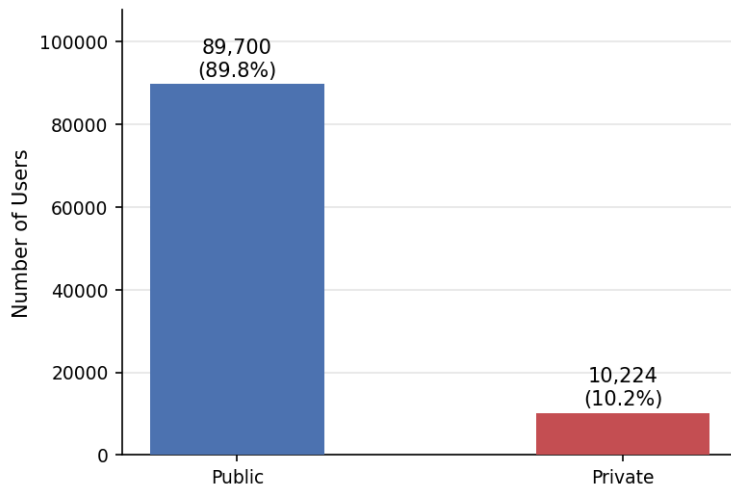


Figure 1: Types of accounts in the crawled users

Users with public accounts do not necessarily expose all of their data, as *Steam* allows fine-grained control over what is visible. Crucially, many users did not permit access to their game libraries. As shown in *Table 1*, game data was unavailable for all private users, and among public users, only 54.9% (49,220) had their game library visible. Since a user’s gaming behavior is essential for training the recommendation model, we removed users without an accessible game library from the dataset, together with their associated connections.

Group	Total	Game data available	Game data unavailable	% unavailable
All Crawled	99,924	49,220	50,704	50.7%
Public	89,700	49,220	40,480	45.1%
Private	10,224	0	10,224	100%

Table 1: Users without game data available by account type

The removal of users without gaming information introduced a new issue: in the resulting graph, the remaining users were no longer all connected in a single component. A total of 18,199 users fell outside of the *Largest Connected Component (LCC)*, while 31,021 users were part of it. To make the later computation of network-derived features feasible, we decided to keep only the users from the *LCC* in the final dataset. In the end, we retained 31% (31,021) of the initially visited users (see *Figure 2*).

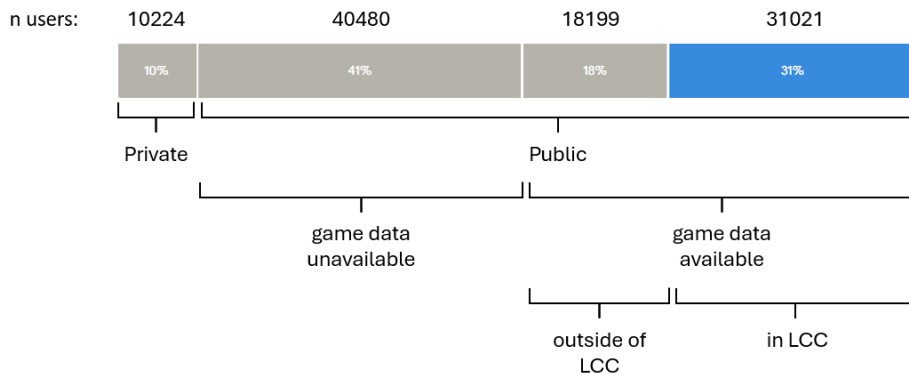


Figure 2: Composition of data based on users’ account type, game data availability, and being part of the LCC

The number of different games owned by the remaining users was 20,918, which is more than twice as few as the initial 45,049 unique games. The largest reduction, however, occurred in the number of friendship edges: from 8,039,785 edges in the raw data to only 40,204 in the final dataset. *Table 2* presents a comparison between the raw data and the final dataset used from now on.

	Scraped Data	Final Data
users	99,924	31,021
friend edges	8,039,785	40,204
games	45,049	20,918
user-game edges	9,193,505	4,010,476

Table 2: Summary of the raw and final data

3.3 Exploratory Data Analysis

To better understand the characteristics of our dataset, we conducted an exploratory data analysis. The results are presented below.

Missing and Unique Values

As shown in Table 3, among the collected attributes, only the user’s country had a notable number of missing values (29.98%). For the remaining attributes, the percentage of missing values was very small: 1.43% for the game’s publisher, 0.63% for the game’s genres, 0.44% for the game’s developer, and 0.33% for the game’s release date. All other attributes were fully available. These results motivated the choice of the imputation strategy used later in the pipeline.

Type of Attribute	Attribute	Missing %
user	country	29.98
	account_created	0
user-game	playtime_minutes	0
game	name	0
	genres	0.63
	developer	0.44
	publisher	1.43
	platforms	0
	release_date	0.33

Table 3: Percentage of missing values across attributes

Table 4 presents the number of unique values for the attributes that were later used as categorical features. The game’s developer and publisher had the highest cardinality (13,906 and 10,624 unique values), followed by the game’s genres (1,161). In contrast, the user’s country had 242 unique values, and the game’s platforms contained only 5. These findings guided the design of the categorical encoding strategy during the training phase.

Type of Attribute	Attribute	Unique Values
user	country	242
game	genres	1,161
	developer	13,906
	publisher	10,624
	platforms	5

Table 4: Count of unique values for attributes used as categorical features

Social Graph Characteristics

After cleaning the dataset, the final user-user friendship network contained 31,021 user nodes connected by 40,204 undirected friendship edges (see Table 5). The median degree was 4, indicating that a typical user had four connections, with degrees ranging from 1 to 184. The degree distribution is highly skewed, as illustrated by the *Complementary Cumulative Distribution Function (CCDF)* in Figure 3. The CCDF plots $P(X \geq k)$ on log-log axes, highlighting the heavy-tailed structure characteristic of social networks, where most users had few friends while a small minority maintained many more connections.

The network was extremely sparse. Its density was only 1.67×10^{-4} , meaning that just 0.017% of all possible user-user connections were present. The average local clustering coefficient was 0.033, implying that on average, 3.3% of a user’s friend pairs were also friends with each other.

To estimate global distances, 500 *BFS* were run from randomly selected seed nodes. The resulting *average shortest path length* of 9.48 suggests that two randomly chosen users were typically 9-10 hops apart, despite the graph’s sparsity. The diameter, computed as the maximum shortest path observed across the *BFS* runs, was 31, which should be interpreted as a lower bound on the true diameter.

Overall, these metrics indicate that the resulting friendship network was heavily tailed and highly sparse, all of which is not surprising considering that it was constructed only from the visited

users encountered during the crawl and further filtered during preprocessing. Consequently, the reported statistics should not be interpreted as representative of the true structure of the full Steam network.

Metric	Value
nodes	31,021
edges (user-user)	40,204
median degree	4
mean degree	5.2
min degree	1
max degree	184
network density	1.67×10^{-4}
average local clustering coefficient	0.0327
connected components	1
approx. average shortest path length	9.4769
diameter	31

Table 5: Social graph characteristics

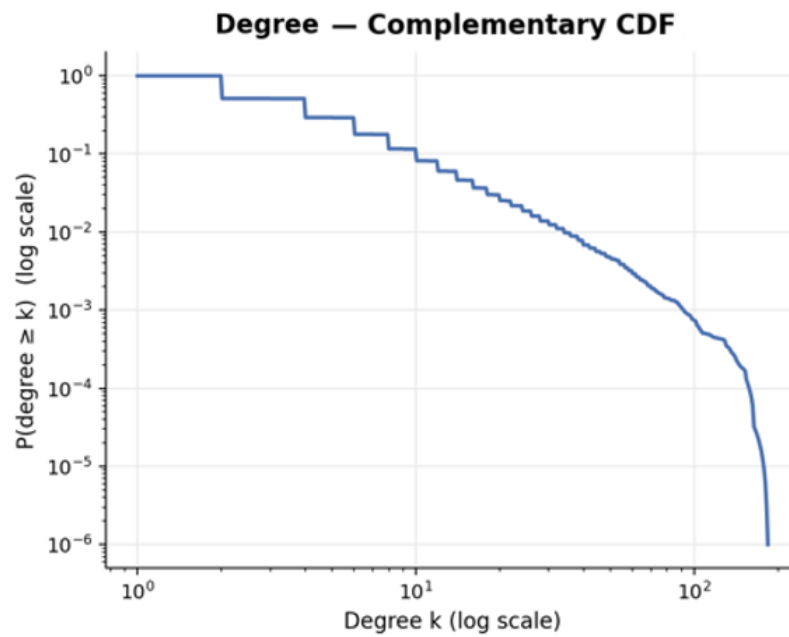


Figure 3: CCDF of degree in the social graph

Bipartite Graph Characteristics

The bipartite user-game graph contained 31,021 user nodes, 20,918 game nodes, and 4,010,476 user-game edges (see Table 6). Its density was 6.18×10^{-3} , higher than that of the user-user friendship network, which is expected since users interact with many more games than people. Nevertheless, the graph remains highly sparse overall, meaning that a typical user owns under 1% of the full game catalogue.

Metric	Value
user nodes	31,021
game nodes	20,918
edges (user-game)	4,010,476
density	6.18×10^{-3}
sparsity	0.993820

Table 6: Bipartite graph characteristic

As shown in Figure 4 and Table 7, users owned on average 129 games, with a median of 45, while a small number of users accumulated exceptionally large libraries, with the maximum reaching 18,381 owned games.

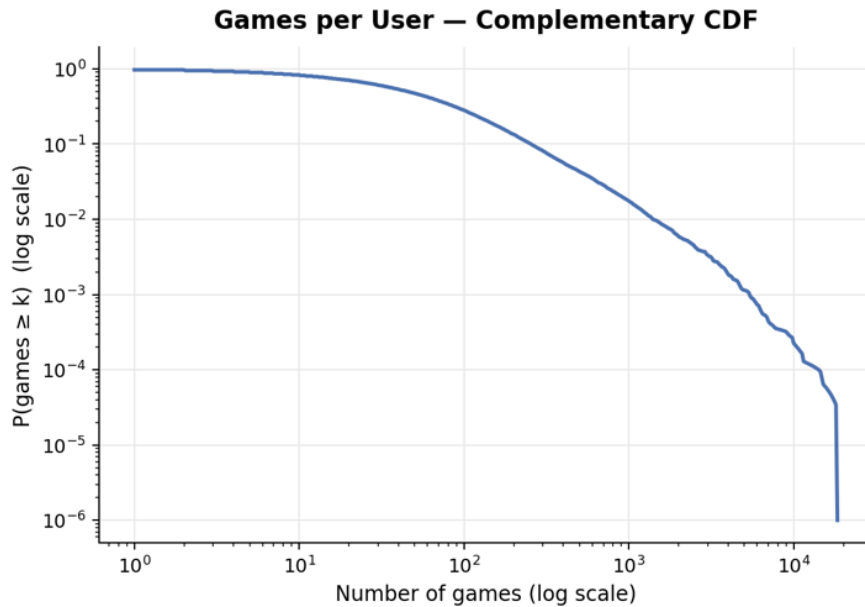


Figure 4: CCDF of games owned per user

Metric	Value
median	45
mean	129
max	18,381

Table 7: Summary statistics of games owned per user

Figure 5 and Table 8 present the opposite perspective: games are owned on average by 192 users, with a median of 30. Again, the distribution exhibits a heavy tail, with a small number of games being owned by thousands of users, and the maximum reaching 24,647.

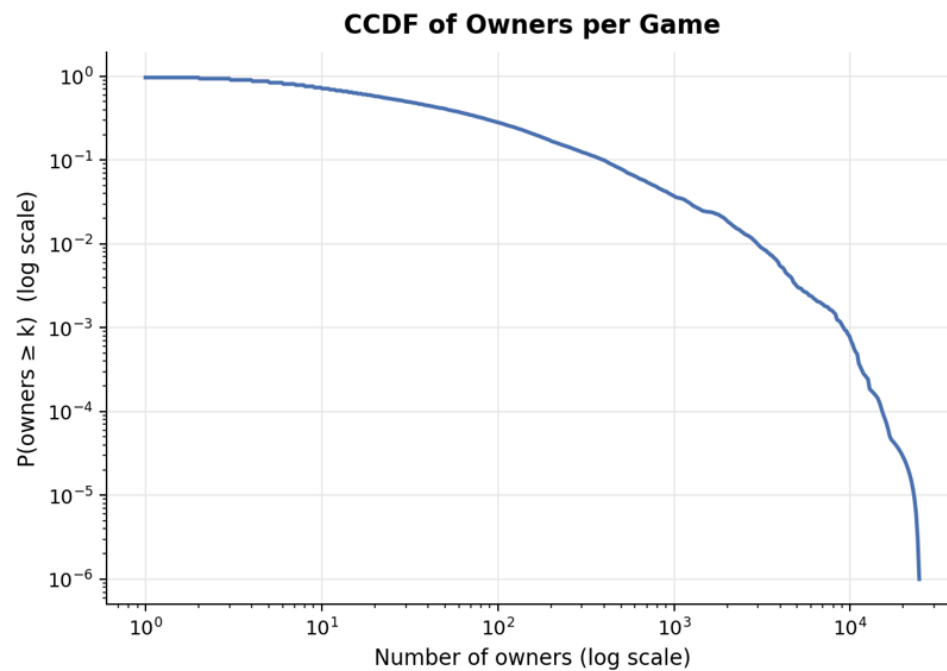


Figure 5: CCDF of the number of owners per game

Metric	Value
median	30
mean	192
max	24,647

Table 8: Summary statistics of the number of owners per game

4 Methodology

4.1 Data Split

To prepare our dataset for machine learning, we combined users and their games into user-game pairs and then split these pairs into train, validation, and test sets. First, we filtered out users who had fewer than five games to ensure that the recommender had enough information to learn their preferences and to make the splitting strategy reliable. Next, we selected two user-game pairs for each user to create validation and test sets. In this *leave-two-out* strategy, the pairs were selected randomly to mimic the overall distribution of user-game interactions (a more realistic approach would use acquisition timestamps, but they were unavailable). The remaining pairs formed the training set (in the training set, the same user appears multiple times). In our initial approach, each user appeared in all three sets, allowing the model to learn the user's taste and then evaluate how well it could recommend games for that same user.

Our datasets contained only positive interactions (user-game pairs that truly existed), so we needed to add negative pairs (user-game pairs that were generated) to allow the model to learn to distinguish which games the user chose to acquire from which they did not. We used negative sampling with the following ratios:

- Training: 1 positive: 10 negatives
- Validation: 1 positive: 100 negatives
- Test: 1 positive: 100 negatives

We intentionally added more negative pairs to the validation and test sets to make the recommendation task more challenging.

Negative pairs were sampled using a mixed strategy: 30% of negatives were chosen from the 30% most popular games (based on the number of users who owned them), with sampling probability proportional to their ownership count. The remaining 70% were sampled uniformly at random from all games in the catalog. This ensured that the recommender learned that popular games are not necessarily owned by every user, and it made the final recommendation task more difficult.

Importantly, all user-game pairs, both positive and negative, were unique across all sets. A game was never sampled as a negative for a user who owned it, and for each user, all negative samples were distinct.

4.2 Features Engineering

Before we could start training our model, we had to prepare a suitable feature set from the final dataset so that the model could learn meaningful patterns. These features were added to each

user-game pair, both positive and negative. Some features, such as the user's country or the game's developer, did not require additional processing and were kept in their original form. The preparation of the remaining features is described below.

Leakage Prevention

All the features were computed using positive user-game pairs **not present** in the validation and test sets. The user-game pairs belonging to the validation and test sets were assumed not to exist during feature engineering. This ensured that no data leakage occurred and that the model could not learn from interactions it would later be evaluated on.

Owned Flag

A column '*owned*' was added, with value *1* for positive pairs and *0* for negative pairs. This column was used during training to indicate which games a user actually owned, and later in validation and testing, it served as the ground-truth label for evaluation. Importantly, the '*owned*' column functioned solely as the supervised target label and was never included among the input features.

Non-Network Features

Playtime Capping

After inspecting our dataset, we discovered that a few users had extremely large playtime values, sometimes even exceeding the amount of time their accounts had existed. To reduce the noise this could introduce, we capped each user's playtime so that it could not exceed the lifetime of their account. To make the approach more robust in cases where the account-creation date was missing, we applied a global cap using the 99.9th percentile of playtime instead. This cleaned playtime value was then used for all further computations.

User-Level Aggregates

The following three features were computed for each user:

- *total_games_owned* - the number of distinct games owned by the user.
- *total_playtime_minutes* - the sum of capped playtime across all owned games.
- *median_playtime_minutes* - the median capped playtime across owned games (the median was used instead of the mean to prevent outliers from disproportionately influencing the value).

Game-Level Aggregates

Two features were computed for each game:

- *user_count* - the number of distinct users who own the game, serving as a proxy for its overall popularity.
- *game_total_playtime_minutes* - the total capped playtime accumulated across all users, capturing the overall engagement with the game.

Genres Features

Steam's genre labels were very noisy. Many genre names referred to the same category but appeared in different languages or used synonyms. To address this, we first mapped all genre labels into broader categories. (see *Figure 6*).

```
genre_groups = {
  "Action": ['Action', 'Acción', 'Акcja', 'Aksiyon', 'Actie', 'Akční', 'Экшены', '動作'],
  "Adventure": ['Adventure', 'Aventura', 'Avontuur', '冒险', 'Приключенческие игры', 'Macera'],
  "RPG": ['RPG', 'RPG', 'Ролевые игры', '角色扮演'],
  "Casual": ['Casual', 'Occasionnel', 'Казуальные игры'],
  "Indie": ['Indie', 'Indépendant', '独立', 'Инди'],
  "Racing": ['Racing', 'Rol'],
  "Simulation": ['Simulation', 'Simuladores', 'Symulacje', 'Симуляторы'],
  "Strategy": ['Strategy', 'Strategie', 'Стратегии'],
  "Sports": ['Sport', 'Sports'],
  "Violent": ['Gore', 'Violent'],
  "Adult": ['Nudity', 'Sexual Content'],
  "Non-gameplay/Tools": ['Education', 'Documentary', 'Movie', 'Tutorial', 'Accounting',
    'Audio Production', 'Video Production', 'Photo Editing',
    'Design & Illustration', 'Web Publishing', 'Utilities',
    'Software Training', 'Game Development', 'Animation & Modeling'],
  "Other": ['Early Access', 'Episodic', 'Free To Play',
    'Massively Multiplayer', 'Бесплатные'],
}
```

Figure 6: Genres groups dictionary

We ended up with 13 genre groups: *Action*, *Adventure*, *RPG*, *Casual*, *Indie*, *Racing*, *Simulation*, *Strategy*, *Sports*, *Violent*, *Adult*, *Non-gameplay Tools*, and *Other* (see the *Appendix Figure 2*).

From this mapping, additional user-level features were derived:

- *unique_genres_played* - the number of distinct genre groups in which the user has played at least one game.
- *user_playtime_group_<genre_name>* - one feature per genre group, representing the user's total capped playtime in that genre. If a game belonged to multiple genre groups, its playtime was split evenly across them to avoid double-counting (the playtime contribution of that game to each genre was divided by the number of genre tags it had).

Release Date

Release date information was originally stored as strings with inconsistent formatting (varying day-month-year orders, full month names, numeric formats, quarter-based representations such as *Q1 2024*, and locale-specific formats including Chinese date notation). To standardize this feature and convert it into a meaningful numerical value, we used the *dateparser* library to handle heterogeneous date formats. Instead of encoding raw strings as categorical variables, we computed the number of months each game had been available on *Steam* as of *14th April 2026*.

Network-Derived Features

Network-derived features, unlike non-network features, were computed using the *Steam* graph. These features capture the social structure of the platform and how users and games relate to one another.

Friend Count

For each user, we computed the number of friendships using the raw scraped data, including connections to discovered (but not visited) users as well as visited users. This ensured that the true number of friends was preserved and not affected by the data-cleaning steps required for machine learning preparation.

As shown in *Table 9*, the median number of friends per user was 97, and the maximum observed value was 1,999. These values are not unusually high, as *Steam* imposes a limit on the number of friends a user can have. The base limit is 250 friends, and it increases by 5 friends for every profile level earned, up to a maximum of 2,000 (*Valve Corporation 2026*). An additional plot presenting the *CCDF* of friend counts per user is provided in the *Appendix Figure 3*.

Metric	Value
median	97
mean	135
max	1,999

Table 9: Summary statistics of friend count per user

Game Embeddings

To compute game embeddings, we used an algorithm developed by *Coscia* for estimating distances on networks (*Coscia 2020*). The method takes as input a *NetworkX* graph constructed from the friendship network and a binary ownership matrix indicating which users own which games. It then computes game-game similarity scores based on distance calculations. The distance measure requires all nodes to lie within the same connected component. That is why users outside the *LCC* were removed during data cleaning.

For each pair of games, their ownership vectors were subtracted, producing a vector encoding agreement and disagreement between users: a value of 0 if both users either owned or did not own the game, and $-1/1$ if one user owned it and the other did not. The final similarity score between two games was computed as:

$$\delta(v_1, v_2, G) = \sqrt{(v_1 - v_2)^\top L^\dagger (v_1 - v_2)}$$

where $L^\dagger$ is the *Moore-Penrose* pseudoinverse of the graph *Laplacian*, which was used as a weight. Disagreements between socially distant users contributed less to the score than disagreements between socially close users. As a result, the measure enriched simple ownership overlaps with information about the structure of the *Steam* social network.

The algorithm outputs a full similarity matrix for all game-game pairs. Since the matrix contained values for more than 20,000 games, it was too large to use directly as features. Following prior work that applied this distance measure as the basis for dimensionality reduction to produce network-aware embeddings (*Damstrup, Madsen & Coscia 2023*), and work demonstrating the approach on cultural data analogous to video games (*Coscia 2024*), we applied *PCA* to obtain compact game representations. *PCA* was chosen for its computational efficiency and interpretability, allowing us to select any desired number of components ordered by explained variance.

First, we used *log1p* transformation on scores (it computes $\log(1 + x)$, which ensures that scores equal to 0 can be handled - since $\log(0)$ would otherwise be undefined) to compress the scale. Next, all scores were standardized using *StandardScaler*, as *PCA* is sensitive to differences in scale. We then applied *PCA*, which produced components explaining the total variance in the similarity matrix (see *Figure 7*).

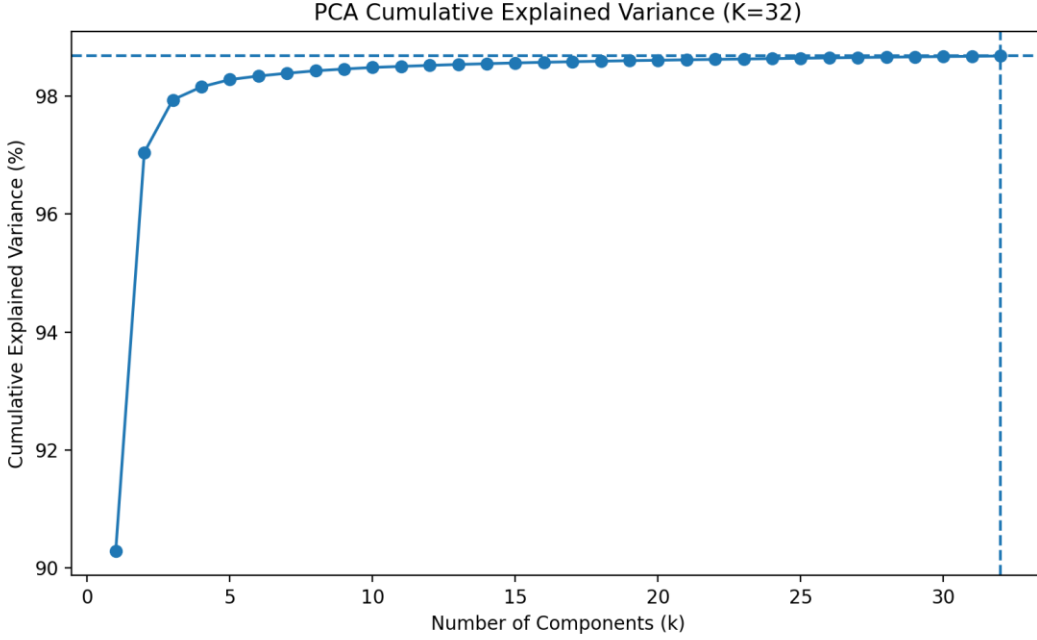


Figure 7: Variance explained by each component

Although the first component (*game_emb_0*) explained 90.29% of the variance, we kept 32 components to better capture the structure of games' similarity (which corresponded to 98.68% of the explained variance). The choice of 32 follows common practice in embedding-based models.

This resulted in 32 embedding features for each game:

game_emb0, game_emb1, ..., game_emb31

which allowed the model to learn how similar different games were based on both ownership patterns and the underlying social network.

Summary Overview of All Features

Table 10 presents all the features we used, separated by either belonging to a user or a game, and whether they were numerical or categorical.

Type of features	Numerical	Categorical
user's features	total_games_owned, total_playtime_minutes, median_playtime_minute, unique_genres_played, user_playtime_group_<genre_name>, friend_count	country
game's features	user_count, game_total_playtime_minutes, release_date, game_emb_<num>	genres, developer, publisher, platforms

Table 10: Summary of all features used

5 Models

5.1 Task Formulation

We tried to mimic how the recommender works on *Steam* - predicting which games the user is likely to interact with. Since the models did not see user-game pairs from the validation and test sets, those pairs were treated as future interactions not yet present. However, because we knew that the positives in the validation and test sets existed, we could use supervised learning; therefore, we formulated the problem as a binary classification task with ranking evaluation. Our target variable was whether a user owns a game:

- $owned = 1 \rightarrow$ user owns the game (positive interaction)
- $owned = 0 \rightarrow$ user does not own the game (negative sample)

For each set of 101 candidate games for a specific user (1 owned game and 100 sampled negatives), the model predicted the probability of ownership:

$$P(owned = 1 \mid user, game)$$

These probabilities were then used as ranking scores, meaning that for each user, candidate games were sorted in descending order of predicted probability. The goal was to teach the models to assign the highest probability to the game the user truly owns.

Although the models were trained as classifiers, their effectiveness was evaluated primarily in terms of ranking quality, which is the main objective of our research.

We trained multiple machine learning models to compare their performance and select the most suitable model for our recommendation task.

5.2 Reference Models

To provide a meaningful baseline for comparison, we evaluated two simple reference models:

- *Random Reference Model*
- *Popularity Reference Model*

These models served as lower-bound performance references for more advanced approaches.

Random Reference Model

The *Random Reference model* assigned a random score to all 101 candidate games for each user independently. The games were then ranked based on those scores. To reduce variance, we repeated the random experiment 100 times and averaged the results across runs.

Popularity Reference Model

Popularity Reference model ranked games based on their global popularity, measured using the *user_count* feature. This means that for each user, candidate games were simply ordered by how many users owned them. This approach completely ignored individual preferences, but unlike the random baseline, it introduced a strong global popularity signal.

5.3 Common Framework

Features variants

All traditional and neural network models (except *LightGCN*) were trained in two versions:

- *Baseline* - only non-network features
- *Network* - extended *baseline* with graph-derived features: 32 game embeddings and *friend_count* for users

The complete list of features is presented in *Table 11*.

Feature Type	Baseline Model	Network Model
Numeric features	• <i>total_games_owned</i> • <i>total_playtime_minutes</i> • <i>median_playtime_minutes</i> • <i>unique_genres_played</i> • <i>user_count</i> • <i>game_total_playtime_minutes</i> • <i>release_date</i> • <i>user_playtime_group_<genre></i>	• <i>total_games_owned</i> • <i>total_playtime_minutes</i> • <i>median_playtime_minutes</i> • <i>unique_genres_played</i> • <i>user_count</i> • <i>game_total_playtime_minutes</i> • <i>release_date</i> • <i>user_playtime_group_<genre></i> • <i>friend_count</i> • <i>game_emb_0 – game_emb_31</i>
Categorical features	• <i>country</i> • <i>genres</i> • <i>developer</i> • <i>publisher</i> • <i>platforms</i>	• <i>country</i> • <i>genres</i> • <i>developer</i> • <i>publisher</i> • <i>platforms</i>
Identifier columns (not used as features)	• <i>steamid</i> • <i>appid</i>	• <i>steamid</i> • <i>appid</i>

Table 11: Complete list of all features used by Baseline and Network models.

Hyperparameter tuning

For each model, we defined a hyperparameter grid, and then those hyperparameters were tuned on the validation set. The best configuration for each model was selected based on the recommendation score.

Multi-seed evaluation

We evaluated all models using three random seeds (0, 1, 42). Each seed-controlled weight initialization, shuffling, and data sampling during model training. All evaluation metrics were aggregated across runs, and the results were reported as *mean ± standard deviation*. By doing this, we ensured that performance differences were not just random variations but reflected the behavior of the model itself.

5.4 Traditional Models

Preprocessing pipeline

All traditional models used a preprocessing pipeline implemented with *sklearn*'s *ColumnTransformer*, which allowed different transformations to be applied to numerical and categorical features in a consistent and automated way.

Numerical features

Missing values in numerical features were handled using *median imputation*, where each missing value is replaced with the median of the corresponding feature. The *median* was chosen because it is robust to outliers, which were present in our dataset.

For *Logistic Regression*, numerical features were additionally scaled with *StandardScaler*. This transformation centered the data around zero and scaled it to have unit variance:

$$x' = (x - \mu)/\sigma$$

where μ is the *mean* of the feature and σ is its *standard deviation*. This ensured that all features were on a comparable scale.

In contrast, tree-based models such as *Random Forest* and *XGBoost* do not require feature scaling. These models make decisions based on threshold splits rather than distances, which makes them inherently insensitive to the scale of the input features (*see the Appendix Figure 4*).

Categorical features

We replaced missing values in categorical features with a constant value '*MISSING*', allowing the model to treat missing data as a separate category rather than ignoring it.

Then we applied *One-Hot Encoding*, which converted each categorical feature into a set of binary variables. For each possible category of a given feature, a separate binary feature was created, taking the value *1* if the instance belongs to that category and *0* otherwise (*see Figure 8*).

country		country_poland	country_germany	country_missing
Poland		1	0	0
Germany	→	0	1	0
Poland		1	0	0
MISSING		0	0	1

Figure 8: Example of One-Hot Encoding applied to categorical features

5.4.1 Logistic Regression

We used *Logistic Regression* as a simple model suitable for a binary classification. It estimated the probability of a user owning the game as a linear combination of input features passed through a sigmoid function. Each feature contributed additively to the final score, weighted by learned coefficients.

The model computed:

$$z = w^T x + b$$

and transforms it using the sigmoid function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

where x is the feature vector describing the user-game pair, w are the learned weights, and b is the bias term. Despite its simplicity, it provided a strong and interpretable benchmark for comparison.

Model configuration

We implemented the model using the *LogisticRegression* class from *scikit-learn*. The final configuration was selected through hyperparameter tuning and used the following settings:

- *solver*: *saga* - suitable for large-scale and sparse data after one-hot encoding
- *penalty*: *L2* - regularizes the model by constraining coefficient magnitudes
- *regularization strength* $C = 1.0$
- *class_weight*: *"balanced"* - compensates for class imbalance by increasing the weight of the minority class

These hyperparameters were found to perform best for both *Baseline* and *Network* variants.

5.4.2 Random Forest

Random Forest is an ensemble learning method based on decision trees. Instead of learning a single decision tree, it constructs a collection of trees and aggregates their predictions. Each tree is trained using bootstrap sampling (a random subset of the training data sampled with replacement). Additionally, at each split, only a random subset of features is considered. This combination of data and feature randomness increases model diversity, which improves robustness and reduces overfitting. Unlike *Logistic Regression*, *Random Forest* can capture nonlinear relationships and interactions between features.

In our setup, *Random Forest* predicted the probability of a user owning a game by averaging the predictions across all trees, and this score was then used for ranking.

Model configuration

The model was implemented using the *RandomForestClassifier* from *scikit-learn*. The final configuration was selected through hyperparameter tuning and used the following settings:

- *n_estimators* = 120 - number of trees in the ensemble
- *max_depth* = 30 - maximum depth of each tree, controlling model complexity
- *min_samples_split* = 20 - minimum number of samples required to split a node
- *min_samples_leaf* = 10 - minimum number of samples required in a leaf node
- *max_features*: "sqrt" - each split considers only a subset of features, increasing diversity among trees
- *class_weight*: "balanced_subsample" - compensates for class imbalance by computing weights separately for each tree

These hyperparameters were found to perform best for both *Baseline* and *Network* variants.

5.4.3 XGBoost

XGBoost is a gradient boosting method based on decision trees. Unlike *Random Forest*, which builds trees independently, *XGBoost* constructs trees sequentially, with each new tree focusing on correcting the errors made by the previous ones.

At each iteration, *XGBoost* adds a new tree that corrects the errors of the current ensemble by reducing the loss function. This boosting approach allows the model to capture complex patterns and interactions between features more effectively.

We trained *XGBoost* to predict the probability of a user owning a game. The model aggregated the contributions of all trees in the ensemble and applied a sigmoid transformation to obtain the

final probability. This probability was then used as the ranking score, similarly to the *Random Forest* setup.

Model configuration

We implemented the model using the *XGBClassifier* from the *XGBoost* library. The final configuration was selected through hyperparameter tuning and used the following settings:

- *n_estimators*: 300 - number of trees built sequentially
- *max_depth*: 6 - maximum depth of each tree, controlling model complexity
- *learning_rate*: 0.1 - scales the contribution of each tree; smaller values lead to more stable learning
- *subsample*: 0.8 - fraction of training data used for each tree, helping reduce overfitting
- *colsample_bytree*: 0.8 - fraction of features used when constructing each tree
- *min_child_weight*: 5 - minimum amount of data required in a leaf
- *reg_lambda*: 2.0 - L2 regularization to limit large weights
- *reg_alpha*: 0.1 - L1 regularization encouraging simpler model

These hyperparameters were found to perform best for both *Baseline* and *Network* variants.

5.5 Neural and Graph Convolutional Models

Preprocessing pipeline

Unlike the preprocessing used for the traditional models, for the neural models, we applied preprocessing once during cache construction to avoid repeating time-consuming computations across all runs and epochs.

Numerical features

Numerical features for the neural models were preprocessed in the same way as for the traditional models: missing values were filled with the median of each feature, and the features were then scaled to help with gradient-based optimization.

Categorical features

For user categorical features, *One-Hot Encoding* was applied in the same way as for the traditional models. However, game categorical features were treated differently. Features such as developer and publisher are multi-valued, which would produce very long, sparse vectors if one-hot encoded. This would significantly increase dimensionality and memory usage, which

was a practical concern given the *RAM* requirements of neural models. Instead, a vocabulary-based embedding approach was used.

For each game categorical feature (*genre*, *developer*, *publisher*, *platforms*), a vocabulary was built by assigning integer indices starting from 2 to each unique value in the training set. Index 0 was reserved for padding, and 1 for values not present in the training set. Each game’s categorical values were then converted using this vocabulary. For example, a game with genres “*Action*; *Strategy*” could be represented as:

$$[2, 3, 0, 0, \dots]$$

Zeros were used for padding to ensure that all games have vectors of the same length for a given feature. These integer sequences were then used to look up embedding vectors.

During training, each categorical value was represented as a vector of fixed size (larger sizes were used for features with more unique values):

- *genres*: 16
- *developer*: 64
- *publisher*: 64
- *platforms*: 4

The embedding values were randomly initialized and updated via backpropagation during training. If a game had multiple values for a feature (e.g., two genres), the final embedding was computed as the average of the embeddings of those values.

As a result, each game received four dense vectors representing its categorical features, which were far more compact and memory efficient than *One-Hot Encoding*.

LightGCN did not use any features, so no preprocessing was required for it.

Loss Function

All neural models used *Bayesian Personalized Ranking (BPR)* loss during training (*Rendle et al. 2009*). We chose this method because it directly optimizes ranking quality. It penalized the model whenever an owned game was assigned a lower score than a non-owned game, rather than simply trying to maximize the score to 1 for owned games and 0 otherwise - an approach that only indirectly promotes correct ranking (like *Binary Cross-Entropy (BCE)* does, for example).

5.5.1 Neural Network

We trained a simple *Multilayer Perceptron (MLP)* as a neural baseline. An *MLP* is a feedforward neural network that transforms input features through a sequence of linear layers followed by nonlinear activation functions. This structure enables the model to capture nonlinear relationships between features, while backpropagation is used during training to update the weights and minimize the loss.

In our implementation, the encoded features were passed through a stack of fully connected layers with *ReLU* activation and dropout for regularization. A final linear layer produced the ranking score used for evaluation.

Model configuration

We implemented the model in *PyTorch*. The final configuration was selected through hyperparameter tuning and used the following settings:

- *hidden_channels*: 256 - size of the hidden representation
- *num_layers*: 4 - number of fully connected layers in the network
- *learning_rate*: $5e-4$ - step size for the optimizer
- training groups sampled per epoch: 200,000 - a random subset of available *BPR* groups used per epoch (each group contained 1 positive and 10 negatives for the same user)

Training was performed for 20 epochs. The best checkpoint was saved and used for the final evaluation.

These hyperparameters were found to perform best for both *Baseline* and *Network* variants.

5.5.2 LightGCN

We included *LightGCN* (He et al., 2020) as a purely collaborative filtering model that relies exclusively on the user-item interaction graph. Unlike the other models in our study, *LightGCN* does not use any features; therefore, it was the only model without separate *Baseline* and *Network* variants. This allowed us to evaluate how well the graph structure alone supports the recommendation task and to compare its performance with feature-based models. *LightGCN* also served as an initial graph-based model before introducing a more complex *GraphSAGE*.

To implement the model, we constructed a bipartite graph from the training data, with users and games as nodes and ownership interactions as edges. *LightGCN* learns embeddings for users and games directly from this graph and is trained as a link prediction model. In contrast to typical *GCN* architectures, *LightGCN* removes all feature transformations and nonlinear activation functions from the propagation layers. This simplification reduces model complexity and was

shown by the authors to improve performance for collaborative filtering tasks (*He et al., 2020*). The final representation of each node was obtained by averaging its embeddings across all propagation layers, allowing the model to capture information from multiple hop distances. User-game pairs were then scored using the *dot product* of their final embeddings.

Model configuration

We implemented the model in *PyTorch*. The final configuration was selected through hyperparameter tuning and used the following settings:

- *hidden_channels*: 256 - size of the hidden representation
- *num_layers*: 1 - number of layers in the network
- *learning_rate*: 0.001 - step size for the optimizer

Training was performed for up to 30 epochs, with early stopping applied after a minimum of 5 epochs if the model did not improve for 5 consecutive epochs.

5.5.3 GraphSAGE

To combine the strengths of the *MLP*, which can learn complex patterns from feature inputs, and *LightGCN*'s ability to represent users and games directly through embeddings learned from the bipartite graph, we trained *GraphSAGE* (*Hamilton et al., 2017*). It was the model that could potentially capture the most detailed relationships, but as a trade-off, it introduced considerably more complexity.

Like *LightGCN*, *GraphSAGE* used the constructed bipartite network to represent users and games with learned embeddings as a link prediction model, but in our implementation, it operated on a heterogeneous graph that additionally included user-user friendship edges. The initial representations were based on user and game features and were then updated during training. *GraphSAGE* also applied feature transformations, learned weight matrices, and nonlinear activation functions. Instead of simply averaging information from neighbors, it learned how to combine the signals coming from them. To produce the final user-game score, a two-layer *MLP* decoder was used.

Model configuration

The model was implemented using *PyTorch Geometric*. The final configuration was selected through hyperparameter tuning and used the following settings:

- *hidden_channels*: 128 - size of the hidden representation
- *num_layers*: 2 - number of graph convolutional layers

- *learning_rate*: $5e-4$ - step size for the optimizer
- *num_neighbors*: 10 - number of neighbors sampled per node at each graph convolution layer during mini-batch training
- training groups sampled per epoch: $200,000$ - a random subset of available *BPR* groups used per epoch (each group contained 1 positive and 10 negatives for the same user)

Training was performed for 20 epochs. The best checkpoint was saved and used for the final evaluation.

These hyperparameters were found to perform best for both *Baseline* and *Network* variants.

6 Results

6.1 Metrics

All metrics that we used were computed per user and then averaged across all users. The ranking metrics were as follows:

- *HitRate@K*, which measured whether the positive game appeared in the top- K recommendations.

$$HitRate@K = \begin{cases} 1, & \text{if the positive game is ranked in top-} K \\ 0, & \text{otherwise} \end{cases}$$

- *NDCG@K* (*Normalized Discounted Cumulative Gain*):

$$DCG@K = \sum_{i=1}^K \frac{y_i}{\log_2(i+1)}$$

$$NDCG@K = \frac{DCG@K}{IDCG@K}$$

where $y_i \in \{0,1\}$ indicates whether the game was at position i was positive, and *IDCG@K* denotes the maximum possible *DCG@K* obtained from an ideal ranking.

In our setting, since there is only one relevant item, *NDCG@K* depends solely on its position in the ranking: a higher rank gives a higher score, while a lower rank gives a lower score.

For $K=1$, the score of *NDCG@1* is equivalent to *HitRate@1*. This is because the ranking contains one position, and no discounting is applied. For $K > 1$, *NDCG* captures ranking quality and differs from the metrics mentioned above.

- *MRR (Mean Reciprocal Rank)*

$$MRR = \frac{1}{|U|} \sum_{u \in U} \frac{1}{rank_u}$$

where $rank_u$ is the position of the positive game for user u .

In the later stages of the analysis, we focused primarily on *MRR*, *HitRate@1*, and *HitRate@10*, as these measures were the most informative. The final model comparison was based on *MRR*, because in our setting with a single positive game per user, capturing how early the positive game appears in the ranking was more meaningful than simply checking whether it appears within the top-K.

Additionally, learning curves were plotted to check if the models generalized well (*see the Appendix Figures 5-15*).

6.2 Models' Results

Model	HitRate@1	HitRate@10	MRR
Random	0.0099	0.0989	0.0514
Popularity	0.1506	0.5879	0.2841
Logistic Regression <i>Baseline</i>	0.1610 ± 0.0000	0.7485 ± 0.0000	0.3384 ± 0.0000
Logistic Regression <i>Network</i>	0.7784 ± 0.0013	0.9445 ± 0.0008	0.8437 ± 0.0006
Random Forest <i>Baseline</i>	0.6241 ± 0.0047	0.8345 ± 0.0017	0.6989 ± 0.0026
Random Forest <i>Network</i>	0.9577 ± 0.0008	0.9781 ± 0.0006	0.9649 ± 0.0007
XGBoost <i>Baseline</i>	0.7768 ± 0.0004	0.8992 ± 0.0016	0.8187 ± 0.0006
XGBoost <i>Network</i>	0.9942 ± 0.0002	0.9976 ± 0.0001	0.9953 ± 0.0001
Neural Network <i>Baseline</i>	0.1935 ± 0.0013	0.6948 ± 0.0022	0.3425 ± 0.0013
Neural Network <i>Network</i>	0.1917 ± 0.0020	0.6976 ± 0.0042	0.3419 ± 0.0017
GraphSAGE <i>Baseline</i>	0.2066 ± 0.0018	0.7408 ± 0.0020	0.3663 ± 0.0018
GraphSAGE <i>Network</i>	0.2101 ± 0.0028	0.7444 ± 0.0022	0.3708 ± 0.0029
LightGCN	0.2806 ± 0.0002	0.7996 ± 0.0007	0.4384 ± 0.0003

Table 12: Comparison of baseline and network models' results with reference models.

Rank	Model	MRR
1	XGBoost <i>Network</i>	0.9953 ± 0.0001
2	Random Forest <i>Network</i>	0.9649 ± 0.0007
3	Logistic Regression <i>Network</i>	0.8437 ± 0.0006
4	XGBoost <i>Baseline</i>	0.8187 ± 0.0006
5	Random Forest <i>Baseline</i>	0.6989 ± 0.0026
6	LightGCN	0.4384 ± 0.0003
7	GraphSAGE <i>Network</i>	0.3708 ± 0.0029
8	GraphSAGE <i>Baseline</i>	0.3663 ± 0.0018
9	Neural Network <i>Baseline</i>	0.3425 ± 0.0013
10	Neural Network <i>Network</i>	0.3419 ± 0.0017
11	Logistic Regression <i>Baseline</i>	0.3384 ± 0.0000
12	Popularity	0.2841
13	Random	0.0514

Table 13: Ranking of the models based on the MRR scores.

Table 12 presents the results for all evaluated models using *HitRate@1*, *HitRate@10*, and *MRR*. All trained models exceeded the performance of the reference Random and Popularity model by a large margin. The highest overall performance was achieved by *XGBoost Network*, which reached an *MRR* of 0.9953 ± 0.0001 , followed by *Random Forest Network* with 0.9649 ± 0.0007 and *Logistic Regression Network* with 0.8437 ± 0.0006 .

For all traditional machine learning models, the network variants outperformed their corresponding baseline variants. The largest relative improvement was observed for *Logistic Regression*, where the *MRR* increased from 0.3384 ± 0.0000 to 0.8437 ± 0.0006 after adding network-derived features.

The neural and graph-based approaches achieved lower overall performance. *Neural Network Network* reached an *MRR* of 0.3419 ± 0.0017 , while *GraphSAGE Network* achieved 0.3708 ± 0.0029 . *LightGCN*, which relied only on the user-game interaction graph without additional features, achieved an *MRR* of 0.4384 ± 0.0003 .

The ranking based on *MRR* shown in *Table 13* further highlights the dominance of the network-enhanced traditional models, with the three highest-ranked approaches being *XGBoost Network*, *Random Forest Network*, and *Logistic Regression Network*.

7 Analysis

The results indicate that the network-derived embeddings were highly effective when combined with traditional machine learning models. *Logistic Regression*, *Random Forest*, and *XGBoost* all achieved higher performance after adding the embedding-based network features and friend count information. *Paired t-tests* performed across all ranking metrics confirmed that the improvements between the *Baseline* and *Network* variants were statistically significant for all traditional models.

Because multiple paired *t-tests* were conducted across several evaluation metrics for each model comparison, the resulting *p-values* were adjusted using the Holm correction method (Holm, 1979). This adjustment reduced the probability of *Type I errors* (false positives) caused by multiple comparisons.

Among all evaluated approaches, *XGBoost* achieved the best overall performance, followed by *Random Forest* and *Logistic Regression*. This suggests that the embeddings captured highly informative relational patterns that could be effectively exploited by traditional models, especially tree-based methods capable of learning nonlinear feature interactions.

XGBoost benefited the most from this setup because gradient-boosted trees are highly effective at modeling nonlinear interactions between features. *Random Forest* also performed strongly, although slightly worse than *XGBoost*, likely because its trees are trained independently instead of sequentially correcting previous errors. *Logistic Regression* improved dramatically after adding embeddings, but its linear nature limited its ability to capture more complex feature relationships.

In contrast, the neural and graph-based models achieved lower performance and did not show statistically significant improvements after adding network-derived features. One likely explanation is the computational complexity of training these models on a very large dataset. Due to memory and runtime constraints, both the *Neural Network* and *GraphSAGE* models required batch-based sampled training instead of processing the full ranking problem simultaneously. This meant that during each epoch, the models observed only sampled subsets of user-game interactions, which likely reduced training effectiveness and limited convergence quality.

GraphSAGE introduced additional complexity because it combined graph propagation with feature learning. Although *GCNs* are theoretically well-suited for recommendation problems, the combination of large-scale graph processing, batching constraints, and limited hardware

resources likely prevented the model from fully utilizing the available information in this experimental setup.

LightGCN achieved better results than the *Neural Network* and *GraphSAGE*, suggesting that the interaction graph itself contained meaningful collaborative filtering information. Because *LightGCN* was restricted to using only the user-game interaction graph, it could not leverage the rich feature information available to the other models.

Overall, the results suggest that in this setting, well-engineered features combined with efficient traditional machine learning models were more effective than more complex neural and graph-based architectures. Based on both predictive performance and computational practicality, *XGBoost* was selected for deeper analysis in the following sections. In addition to achieving the highest ranking quality, the model was relatively fast to train, stable across random seeds, and easier to interpret compared to the neural and graph-based approaches.

7.1 Fresh-User Simulation

To verify whether the strong performance of the *XGBoost* model was influenced by user-level data leakage and to check if the model could generalize well to unseen users, we conducted an additional simulation with a stricter evaluation setup.

Experimental Setup

In the original split, the same users appeared across training, validation, and test sets. While this is common in recommender systems, it might have allowed the model to implicitly memorize user-specific patterns.

To check if it happened, we constructed a *fresh-user split*. We first randomly selected 2000 users, and then we removed them from the training set. Those selected users were the only users kept in the validation and test sets. As a result, the model was evaluated on users it had never seen during training.

Results

The results indicate that the model’s performance was not driven by user-level data leakage between the training and evaluation sets (see Tables 14 and 15). Even when evaluated on completely unseen users, the performance remained nearly unchanged, demonstrating strong generalization capabilities. This suggests that the model was learning meaningful relationships between users and games. Notably, the *Network* model maintains extremely high performance under this stricter setup, further confirming the robustness of the learned representations.

The same *paired t-test* procedure with *Holm correction* was also applied in this simulation study. For the *Baseline* model, no statistically significant differences were observed between the original and *fresh-user simulation* on either the validation or test sets. For the *Network* model, no significant differences were found on the test set, although three validation metrics showed statistically significant but very small decreases.

Overall, the simulation did not produce a meaningful performance drop, suggesting that the model generalized well to unseen users rather than relying on user-level leakage.

Metric	XGB Baseline	Simulation Baseline	Δ
HitRate@1	0.7768 ± 0.0004	0.7785 ± 0.0003	+0.0017
HitRate@5	0.8630 ± 0.0021	0.8610 ± 0.0268	-0.0020
HitRate@10	0.8992 ± 0.0016	0.8952 ± 0.0037	-0.0040
MRR	0.8187 ± 0.0006	0.8191 ± 0.0025	+0.0004

Table 14: Comparison between XGBoost Baseline and its simulation.

Metric	XGB Network	Simulation Network	Δ
HitRate@1	0.9942 ± 0.0002	0.9946 ± 0.0002	+0.0004
HitRate@5	0.9965 ± 0.0001	0.9960 ± 0.0000	-0.0005
HitRate@10	0.9976 ± 0.0001	0.9973 ± 0.0002	-0.0003
MRR	0.9953 ± 0.0001	0.9954 ± 0.0001	+0.0001

Table 15: Comparison between XGBoost Network and its simulation.

7.2 Feature Importance and SHAP Analysis

To better understand the behavior of the *XGBoost* models, we conducted a feature importance analysis using *SHAP* (*Shapley Additive exPlanations*) (Lundberg & Lee, 2017). The goal was to identify which features drove the model predictions and to explain the significant performance gain observed after introducing the embedding features.

We compared two models: the *Baseline* model and its extended version with the *game_emb_0* feature. We focused on *game_emb_0* because it explained 90.29% of the variance among the

embedding features and was computationally efficient while still being sufficient to clearly illustrate the shift in model behavior after introducing the embeddings.

The *Baseline* model was strongly dominated by the *user_count* feature, which accounted for over 70% of the total importance. This indicates that the model relied primarily on the popularity signals. Other features, such as *game_total_playtime_minutes* and *release_date*, played a secondary role (see Table 16).

Rank	Feature	mean_abs_shap	Share	Top Interaction
1	user_count	12.159643	73.26%	release_date
2	game_total_playtime_minutes	2.465470	14.85%	user_count
3	release_date	0.661086	3.98%	user_count
4	user_playtime_group_Other	0.149454	0.9%	unique_genres_played
5	total_games_owned	0.149335	0.9%	user_count

Table 16: Feature importance for XGBoost Baseline

After adding *game_emb_0*, the importance distribution changed (see Table 17). The embedding became the second most important feature, contributing to over 30% of the total importance. At the same time, the relative importance of *user_count* decreased, although it still remained the dominant feature.

Rank	Feature	mean_abs_shap	Share	Top Interaction
1	user_count	16.350816	61.12%	game_emb_0
2	game_emb_0	8.113760	30.33%	user_count
3	release_date	0.782508	2.92%	game_emb_0
4	game_total_playtime_minute	0.566819	2.12%	game_emb_0
5	median_playtime_minutes	0.230689	0.86%	release_date

Table 17: Feature importance for XGBoost Baseline with *game_emb_0*

The *SHAP* summary plots (See Figures 9 and 10) visualize both the importance and direction of feature influence. Features are ordered by their overall contribution to the model output. Each point represents a single prediction, while the horizontal position indicates whether the feature

increased or decreased the recommendation score. The color gradient represents the feature value, ranging from low values (blue) to high values (red).

Taken together, these plots clearly show how the model’s behavior changed between the two setups. In the *Baseline* model, the prediction behavior was almost entirely driven by *user_count*, with most other variables having only minor influence on the model’s output. After introducing *game_emb_0*, the embedding contributed strongly alongside *user_count*, indicating that the model no longer relied exclusively on raw popularity signals.

We also observed strong interactions between *game_emb_0* and *release_date* and between *game_emb_0* and *game_total_playtime_minutes*, suggesting that the embedding provided information that complemented these non-network features. This is consistent with how the embeddings were constructed, as they encoded each game’s position within the social ownership graph, placing games owned by similar groups of connected users closer together in the embedding space. Games released around the same period often share similar player communities and popularity trends, while games with high total playtime tend to have large and active player bases. Because *game_emb_0* partially reflected the same patterns, it captured an overlapping but still complementary structure that the non-network features alone could not represent.

Overall, the results show that *game_emb_0* added information that *user_count* and other non-network features could not supply, contributing meaningfully to the model’s performance.

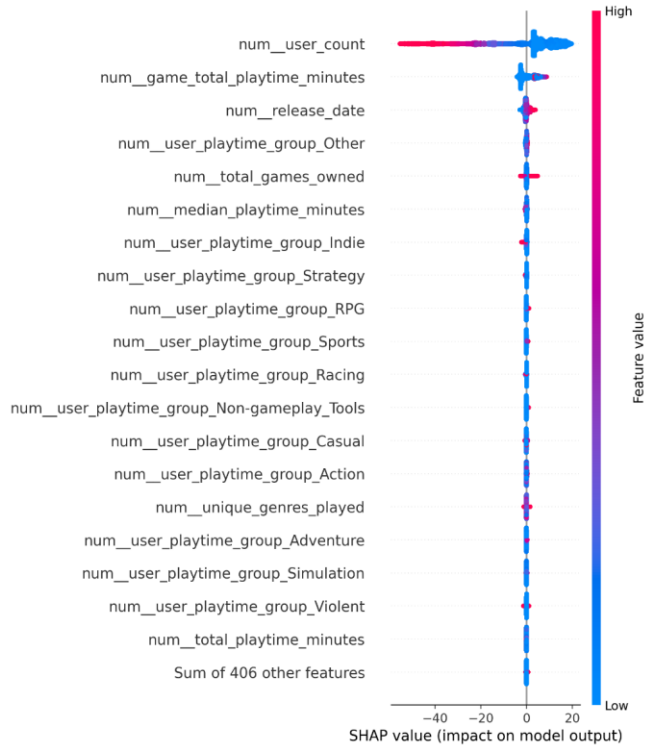


Figure 9: SHAP summary plot for the XGBoost Baseline

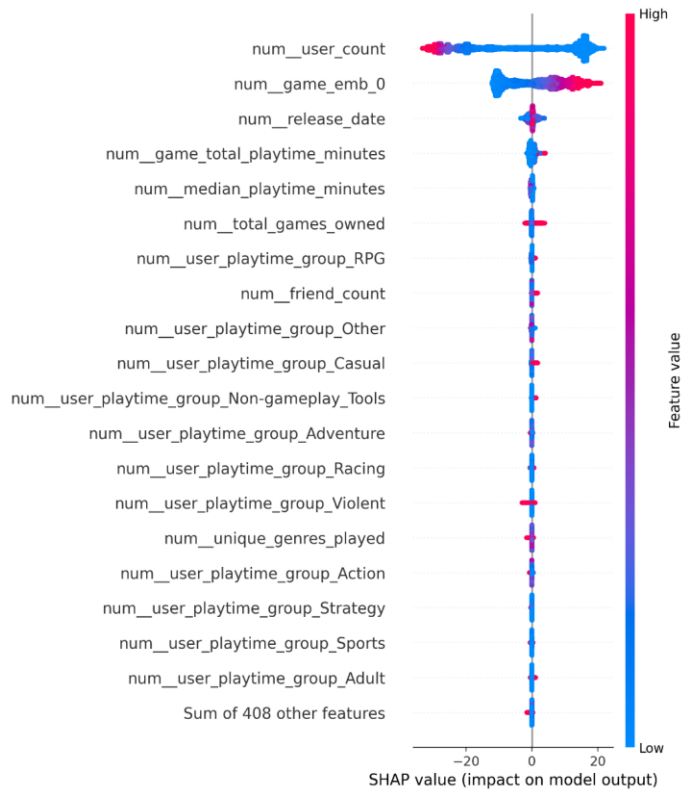


Figure 10: SHAP summary plot for the XGBoost Baseline with game_emb_0.

7.3 XGBoost Experiments

To clarify how *user_count* and *game_embedding_0* influenced the model’s performance, we designed a set of controlled experiments targeting these two features. Building on the *XGBoost Baseline* and *Network* models, we constructed variants with systematically altered feature spaces, allowing us to isolate the direct contribution of each feature and examine their interaction.

We evaluated each experiment using the same multi-seed setup as in the main experiments. To assess statistical significance, we performed *paired t-tests* across all ranking metrics and applied *Holm correction* to account for multiple comparisons.

XGBoost Baseline with *game_emb_0*

First, we extended the *Baseline* model by adding a single embedding dimension (*game_emb_0*). This resulted in a large improvement: *HitRate@1* increased from ~ 0.77 to ~ 0.98 , and *MRR* increased from ~ 0.82 to ~ 0.99 (see Table 18). This result shows that a single embedding already captured nearly all of the graph signal beneficial for ranking. Because *game_emb_0* was the first *PCA* component and explained over 90% of the variance, it effectively concentrated the most information out of the network-derived features.

Model	HitRate@1	HitRate@10	MRR
XGBoost Baseline	0.7768 ± 0.0004	0.8992 ± 0.0016	0.8187 ± 0.0006
XGBoost Baseline with <i>game_emb_0</i>	0.9840 ± 0.0003	0.9936 ± 0.0000	0.9872 ± 0.0003
XGBoost Network	0.9942 ± 0.0002	0.9976 ± 0.0001	0.9953 ± 0.0001

Table 18: Comparison of XGBoost Baseline with *game_emb_0*, XGBoost Baseline, and XGBoost Network

A *paired t-test* comparing the original *XGBoost Baseline* model with the *XGBoost Baseline with game_emb_0* variant showed statistically significant improvements across all evaluated ranking metrics after *Holm correction*.

XGBoost with embeddings only

Secondly, we trained a model using only embedding features. The performance was very low (*HitRate@1* ≈ 0.106 , *MRR* ≈ 0.234), showing that embeddings alone were insufficient for accurate recommendation (see Table 19). This experiment suggests that without user context and raw popularity signals, the model could not rank games effectively.

Model	HitRate@1	HitRate@10	MRR
XGBoost Baseline	0.7768 $\pm$ 0.0004	0.8992 $\pm$ 0.0016	0.8187 $\pm$ 0.0006
XGBoost with embeddings only	0.1062 $\pm$ 0.0000	0.5092 $\pm$ 0.0000	0.2343 $\pm$ 0.0000
XGBoost Network	0.9942 $\pm$ 0.0002	0.9976 $\pm$ 0.0001	0.9953 $\pm$ 0.0001

Table 19: Comparison of XGBoost embeddings only, XGBoost Baseline, and XGBoost Network

A paired *t*-test comparing the *embeddings-only* model with the original XGBoost Baseline confirmed that the *embeddings-only* model performed significantly worse across all evaluated ranking metrics after *Holm correction*.

XGBoost with embeddings and basic user information

As a follow-up step, we created an XGBoost model with embeddings together with a minimal set of user-level features. We used *unique_genres_played* and *total_playtime_minutes*. The goal of this experiment was to test whether the *embedding model* would improve after adding some minimal user context.

The results indicated a noticeable improvement over the *embeddings-only* model. However, the performance was still far below the *Baseline* model (see Table 20). This is consistent with our *SHAP* analysis, which indicated that *game_emb_0* interacted primarily with *user_count*, *release_date*, and *game_total_playtime_minutes*, whereas the user-level features we chose for this experiment had very low importance and no strong interactions.

Model	HitRate@1	HitRate@10	MRR
XGBoost Baseline	0.7768 $\pm$ 0.0004	0.8992 $\pm$ 0.0016	0.8187 $\pm$ 0.0006
XGBoost with embeddings only	0.1062 $\pm$ 0.0000	0.5092 $\pm$ 0.0000	0.2343 $\pm$ 0.0000
XGBoost with embeddings and basic user information	0.1582 $\pm$ 0.0004	0.6114 $\pm$ 0.0002	0.2946 $\pm$ 0.0002
XGBoost Network	0.9942 $\pm$ 0.0002	0.9976 $\pm$ 0.0001	0.9953 $\pm$ 0.0001

Table 20: Comparison of XGBoost with embeddings only, XGBoost with embeddings and basic user information, XGBoost Baseline, and XGBoost Network

The *paired t-test* showed that adding basic user information to the *embeddings-only* model significantly improved performance across all evaluated ranking metrics after *Holm correction*.

XGBoost Baseline without *user_count*

In the following step, we removed the main popularity feature (*user_count*) from the *Baseline* model. The performance decreased dramatically. *HitRate@1* dropped to ~ 0.18 , and *MRR* dropped to ~ 0.31 (see Table 21). This result was consistent with the *SHAP* analysis, which showed that the model depends heavily on the *user_count* feature as a popularity signal.

Model	HitRate@1	HitRate@10	MRR
XGBoost Baseline	0.7768 ± 0.0004	0.8992 ± 0.0016	0.8187 ± 0.0006
XGBoost Baseline without <i>user_count</i>	0.1803 ± 0.0011	0.6269 ± 0.0010	0.3148 ± 0.0009
XGBoost Network	0.9942 ± 0.0002	0.9976 ± 0.0001	0.9953 ± 0.0001

Table 21: Comparison of XGBoost Baseline without *user_count*, XGBoost Baseline, and XGBoost Network

A *paired t-test* comparing the original XGBoost *Baseline* model with the *Baseline* without the *user_count* variant confirmed a statistically significant performance decrease across all evaluated ranking metrics after *Holm correction*.

XGBoost Baseline without *user_count* with *game_emb_0*

Next, we added the embedding (*game_emb_0*) while still excluding *user_count*. The improvement was minimal: *HitRate@1* increased only slightly (~ 0.18 to ~ 0.19), and *MRR* (~ 0.31 to ~ 0.33) also barely increased (see Table 22).

Although the improvement was statistically significant (comparing the *Baseline* without *user_count* model with the corresponding *Baseline* without *user_count* but with *game_emb_0*) as shown by a *paired t-test* with *Holm correction*, the overall performance remained below the original *Baseline* and *Network* models. This indicates that *game_emb_0* alone could not replace the popularity signal provided by *user_count*.

Model	HitRate@1	HitRate@10	MRR
XGBoost Baseline	0.7768 ± 0.0004	0.8992 ± 0.0016	0.8187 ± 0.0006
XGBoost Baseline without <i>user_count</i>	0.1803 ± 0.0011	0.6269 ± 0.0010	0.3148 ± 0.0009
XGBoost Baseline without <i>user_count</i> with <i>game_emb_0</i>	0.1890 ± 0.0007	0.6494 ± 0.0012	0.3269 ± 0.0003
XGBoost Network	0.9942 ± 0.0002	0.9976 ± 0.0001	0.9953 ± 0.0001

Table 22: Comparison of XGBoost Baseline without *user_count* with *game_emb_0*, Baseline without *user_count*, XGBoost Baseline, and XGBoost Network

XGBoost with *user_count* and *game_emb_0*

To further examine the interaction, we trained a minimal model using only two features: *user_count* and *game_emb_0*. Despite the simplicity, the model achieved very high performance with $\text{HitRate}@1 \approx 0.95$ and $\text{MRR} \approx 0.96$ (see Table 23). This experiment demonstrates that most of the predictive power came from the interaction between these two features.

A paired *t*-test comparing the original Baseline model with the minimal *user_count* and *game_emb_0* model showed statistically significant improvements across all evaluated metrics after Holm correction. However, a second paired *t*-test comparing the same minimal model with the XGBoost Network model showed that the Network variant still achieved statistically significantly better performance across all evaluated metrics after Holm correction.

Model	HitRate@1	HitRate@10	MRR
XGBoost Baseline	0.7768 ± 0.0004	0.8992 ± 0.0016	0.8187 ± 0.0006
XGBoost with <i>user_count</i> and <i>game_emb_0</i>	0.9529 ± 0.0000	0.9878 ± 0.0000	0.9647 ± 0.0000
XGBoost Network	0.9942 ± 0.0002	0.9976 ± 0.0001	0.9953 ± 0.0001

Table 23: Comparison of the XGBoost with *user_count* and *game_emb_0*, XGBoost Baseline, and XGBoost Network

XGBoost with user_count

Lastly, we trained an *XGBoost* with only *user_count*. Methodologically, this setup differed from the *Popularity Reference Model*. The *Popularity* model ranked games directly by their popularity (*user_count*), meaning that every user received exactly the same ordering of candidate games. In contrast, the *XGBoost* model treated *user_count* as an input feature within a supervised learning framework. Instead of using the raw popularity values directly as ranking scores, the model learned nonlinear decision boundaries and optimized ownership prediction through gradient boosting.

Despite this additional modeling flexibility, the results remained very similar to the simple *Popularity Reference Model*. The difference in *HitRate@1* was only -0.0034 , while *MRR* slightly increased by 0.0264 from the *reference model* (see Table 24).

Model	HitRate@1	HitRate@10	MRR
XGBoost Baseline	0.7768 ± 0.0004	0.8992 ± 0.0016	0.8187 ± 0.0006
XGBoost with <i>user_count</i> only	0.1472 ± 0.0000	0.5097 ± 0.0000	0.2576 ± 0.0000
Popularity reference model	0.1506	0.5879	0.2840
XGBoost Network	0.9942 ± 0.0002	0.9976 ± 0.0001	0.9953 ± 0.0001

Table 24: Comparison of the *XGBoost with user_count*, *Popularity Reference Model*, *XGBoost Baseline*, and *XGBoost Network*

The *paired t-test* showed that the *XGBoost user_count*-only model performed significantly worse than the *XGBoost Baseline* across all evaluated ranking metrics after *Holm correction*.

Summary

The experiments consistently showed that the performance gains were driven by the interaction between popularity (*user_count*) and the network-derived feature (*game_emb_0*) (see Table 25).

While embeddings alone achieved performance of *HitRate@1* ≈ 0.11 , and only *user_count* *HitRate@1* ≈ 0.15 , their combination achieved extremely high recommendation quality (*HitRate@1* ≈ 0.95). This confirms that the interaction between popularity information and embedding-based game representation provided a compact and highly effective representation of the data.

Rank	Model	HitRate@1	HitRate@10	MRR
1	XGBoost Network	0.9942 $\pm$ 0.0002	0.9976 $\pm$ 0.0001	0.9953 $\pm$ 0.0001
2	XGBoost Baseline with <i>game_emb_0</i>	0.9840 $\pm$ 0.0003	0.9936 $\pm$ 0.0000	0.9872 $\pm$ 0.0003
3	XGBoost with <i>user_count</i> and <i>game_emb_0</i>	0.9529 $\pm$ 0.0000	0.9878 $\pm$ 0.0000	0.9647 $\pm$ 0.0000
4	XGBoost Baseline	0.7768 $\pm$ 0.0004	0.8992 $\pm$ 0.0016	0.8187 $\pm$ 0.0006
5	XGBoost Baseline without <i>user_count</i> with <i>game_emb_0</i>	0.1890 $\pm$ 0.0007	0.6494 $\pm$ 0.0012	0.3269 $\pm$ 0.0003
6	XGBoost Baseline without <i>user_count</i>	0.1803 $\pm$ 0.0011	0.6269 $\pm$ 0.0010	0.3148 $\pm$ 0.0009
7	XGBoost with embeddings and basic user info	0.1582 $\pm$ 0.0004	0.6114 $\pm$ 0.0002	0.2946 $\pm$ 0.0002
8	XGBoost <i>user_count</i> only	0.1472 $\pm$ 0.0000	0.5097 $\pm$ 0.0000	0.2576 $\pm$ 0.0000
9	XGBoost embeddings only	0.1062 $\pm$ 0.0000	0.5092 $\pm$ 0.0000	0.2343 $\pm$ 0.0000

Table 25: Ranking by MRR of XGBoost experiments compared with Baseline and Network models

7.4 Impact of Embeddings on the Model’s Performance

To further understand how the network-derived features influenced the model's performance, we undertook a deeper analysis focusing on the impact of the *game_emb_0* feature. Table 26 presents the main findings, which are further discussed below. All score comparisons in this section are between two variants of our best-performing model: the *XGBoost Baseline* (seed 42) and the *XGBoost Baseline with game_emb_0*. Additional plots can be found in the Appendix (see Figures 16, and 17).

Metric	Value
score separation (<i>XGBoost Baseline</i>)	0.5914
score separation (<i>XGBoost Baseline with game_emb_0</i>)	0.9480
mean top-10 overlap	1.7645
mean rank (<i>XGBoost Baseline</i>)	4.4862
mean rank (<i>XGBoost Baseline with game_emb_0</i>)	1.2590
percentage of users with improved owned game rank	22.33%
Spearman: <i>game_emb_0</i> vs. user count	0.9767

Table 26: Metric used to inspect *game_emb_0* influence on the model's performance

7.4.1 Score Separation

One of the most informative properties of a recommendation model is how confidently it distinguishes relevant from irrelevant items. To assess this in our setting, we computed score separation for both models, defined as:

$$\text{score separation} = \text{mean score}(\text{owned} = 1) - \text{mean score}(\text{owned} = 0)$$

The score separation for the *XGBoost Baseline* was *0.5914*, whereas for *XGBoost with game_emb_0* it was *0.9480* (see Table 26). This shows that adding the first embedding made the model give more confident predictions. This is further illustrated in Figures 11 and 12.

Before adding *game_emb_0*, the score distributions for owned and non-owned games already occupied opposite ends of the score range, but they still had noticeable overlap. For owned games, the distribution extended all the way down to a predicted score of 0, and for non-owned games, it had a long right tail reaching around 0.2.

In contrast, after adding *game_emb_0*, the predicted scores are almost fully centered around 1 for owned games and around 0 for non-owned games. This suggests that the embedding provided a strong discriminative signal that the model using only non-network features did not capture.

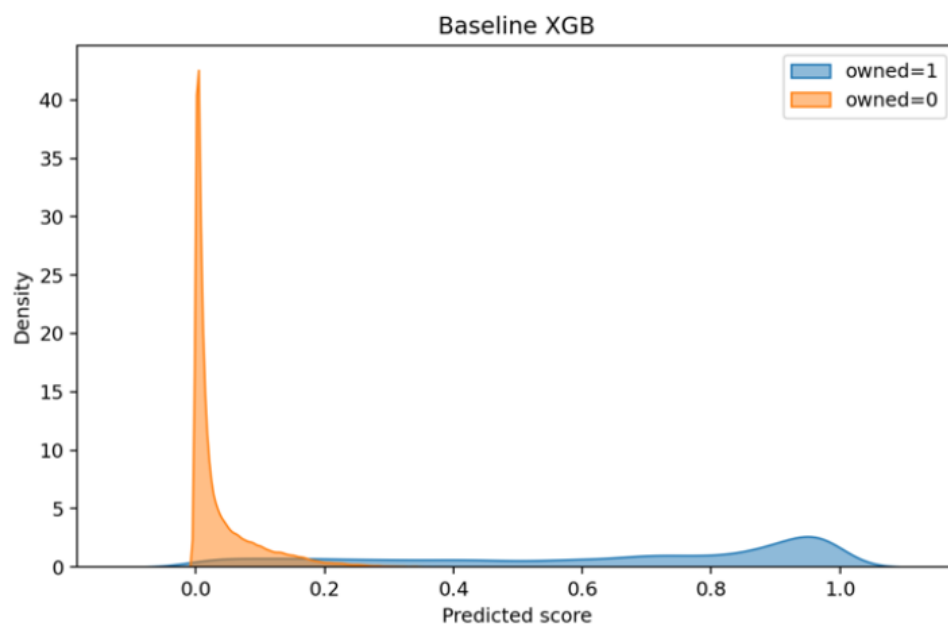


Figure 11: Distribution of predicted scores for the baseline model

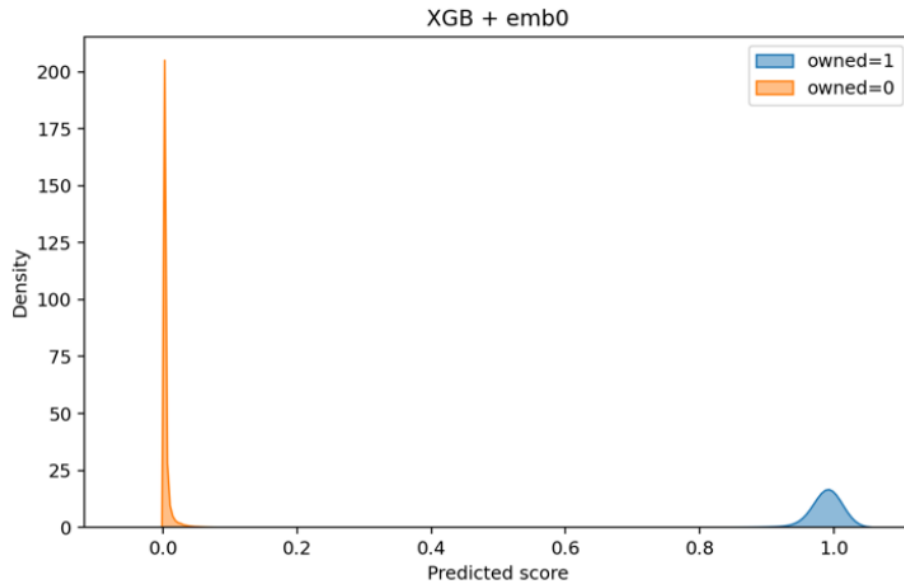


Figure 12: Distribution of predicted scores for the model with `game_emb_0`

7.4.2 Ranking Quality

As shown in Figure 13, the games ranked highest by the models differed. On average, only 1.76 games were the same in the *Top-10* recommended games for both models (see Table 26). This means that the network model did not simply re-weight the existing scores but instead recommended different games.

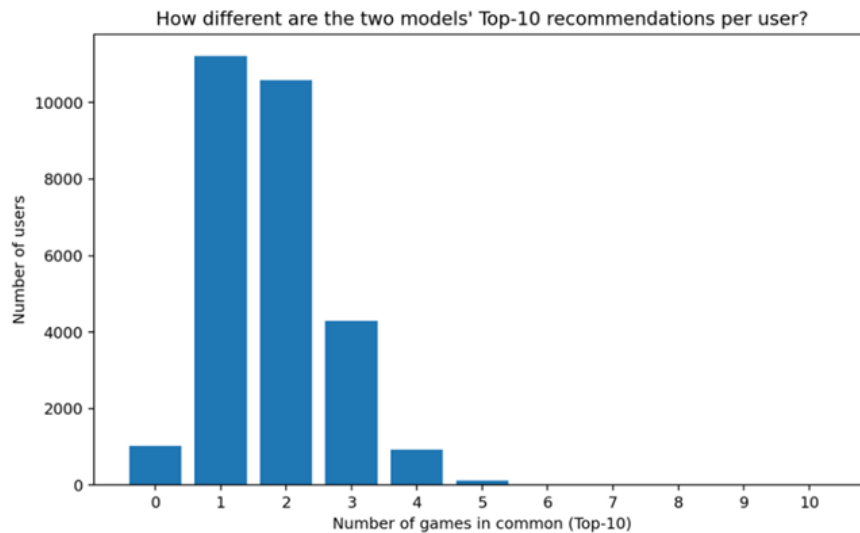


Figure 13: The difference in the baseline model's and the model's with `emb0` Top-10 recommendations per user

What is more, the mean rank of the actually owned game (the position where the owned game appeared in the list) decreased from 4.49 for the *XGBoost Baseline* to 1.26 for the *XGBoost* model with *game_emb_0* (see Table 26). Additionally, 22.33% of users had their owned game ranked higher by the *game_emb_0* model than by the baseline. This is a great improvement, considering that the *Baseline* model already ranked the owned game at position 1 for 77.63% of users (*HitRate@1* = 0.7763). This means that only the remaining 22.37% of users could improve at all, and almost all of them (22.33% out of 22.37%) did.

7.4.3 What Causes the Score Change

To better understand what caused the change in scores, we checked the *Spearman correlation* between the per-game mean Δ score (*Network* model score minus *Baseline* model score) and various network properties (see Figures 14 and 15). The properties used in this analysis are described in Table 27.

For owned games (*owned=1*), most correlations were modest. Popularity-related features, such as *user_count* (+0.200) and *2nd-order neighborhood size* (+0.162), showed modest positive correlations. This means that the model with *game_emb_0* increased the scores more for popular games that the user owned. In contrast, features reflecting social connectivity, *mean owner library size* (-0.272), *mean owner friend count* (-0.315), and *clustering coefficient* (-0.127), showed slight negative correlations. This suggests that games owned by users with richer social and gaming profiles received a smaller boost.

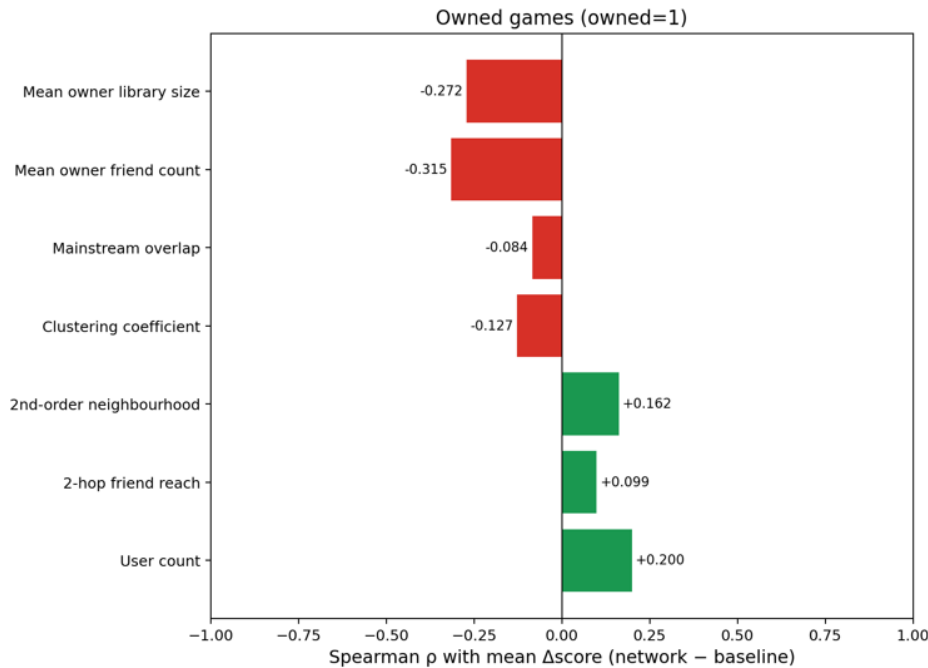


Figure 14: Graph structure correlation with score changes for owned games

For not-owned games, the correlations are stronger. Popularity-related features, *user_count* (-0.755) and *2-hop friend reach* (-0.753), are all strongly correlated with $\Delta score$. This indicates that the model with *game_emb_0* suppressed popular games that a user did not own, which is a desired behavior, as it shows that *game_emb_0* helped differentiate overall popularity from specific user preference. On the other hand, features reflecting social connectivity, such as *mean owner library size* (+0.569) and *mean owner friend count* (+0.419), were positively correlated with $\Delta score$. This shows that the model with *game_emb_0* penalizes less harshly the games owned by users with many games and friends. This may suggest that for those games, the model was less certain about its predictions.

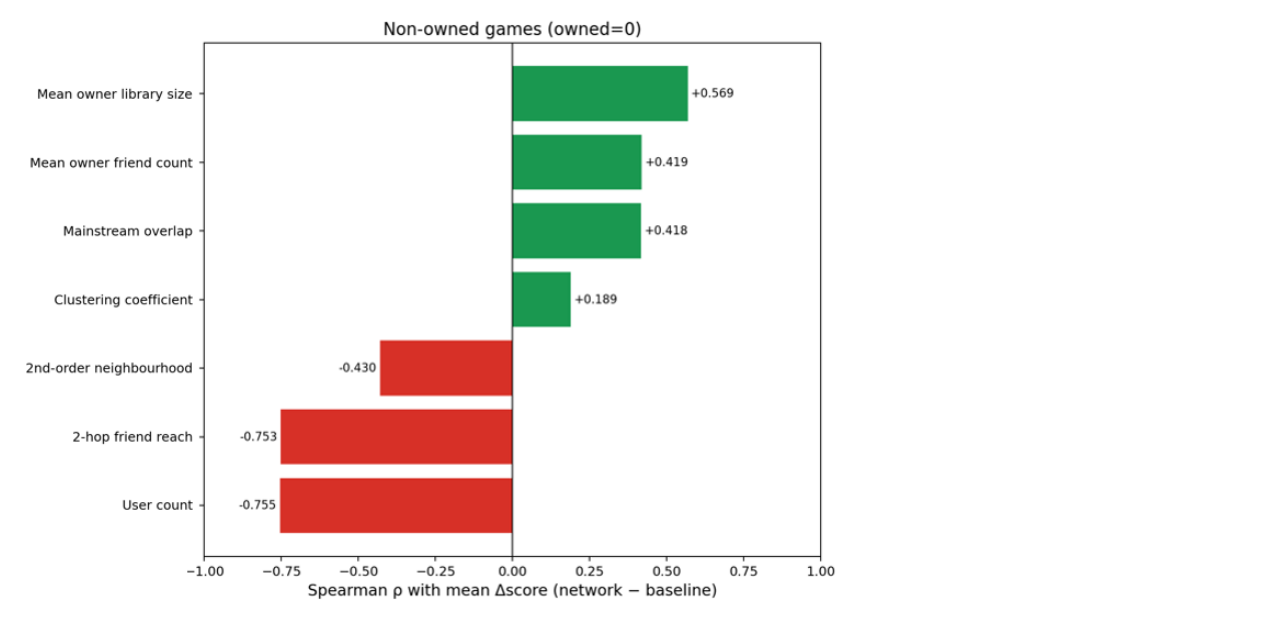


Figure 15: Graph structure correlation with score changes for not-owned games

Network Property	Explanation
mean owner library size	average number of games owned by users who own this game
mean owner friend count	average number of Steam friends among this game’s owners
mainstream overlap	proportion of this game’s owners who also own at least one of the top-10 most popular games
clustering coefficient	how often games that share owners with this game also share owners with each other
2nd-order neighborhood size	number of other games that share at least one owner with this game
2-hop friend reach	number of unique users reachable within two friendship hops from this game’s owners
user count	number of users who own the game

Table 27: Network properties and their explanations

7.4.4 Genre-Level Impact

Figures 16 and 17 show the median $\Delta score$ per game for different genre groups. For owned games, the score was boosted across all genres. It ranged from approximately $+0.36$ (*Adult*) to $+0.63$ (*Strategy*). This suggests that the model became more confident about a user owning the game they truly owned, regardless of genre.

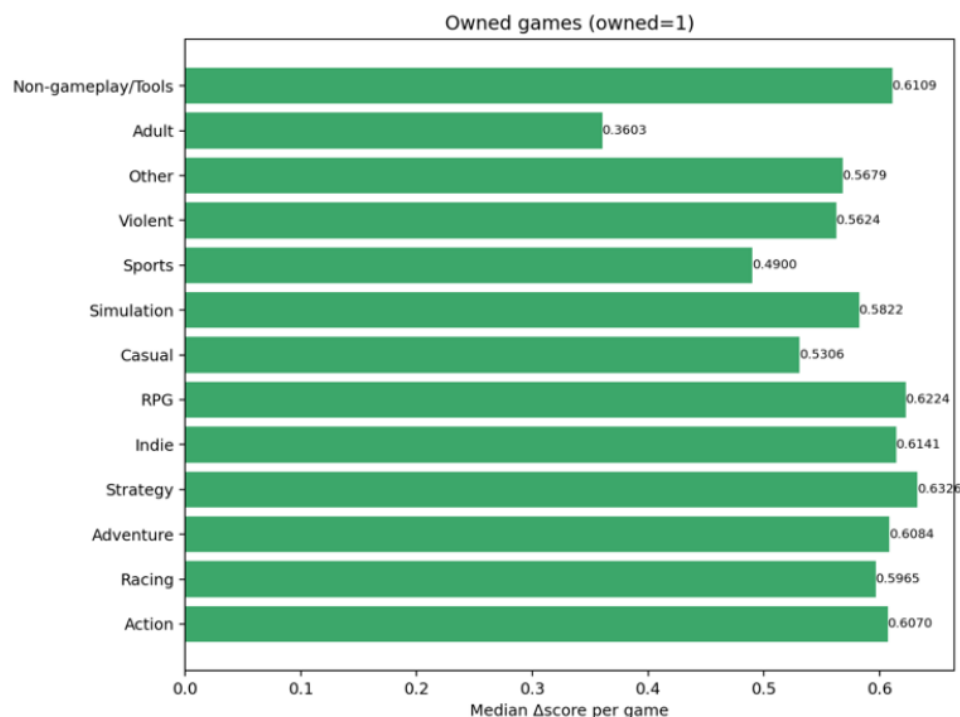


Figure 16: Median change in score per game for each genre for owned games

The median $\Delta score$ per game for genre groups for not-owned games was negative across all genres, with noticeable differences. *Action* (-0.0033), *Racing* (-0.0030), and *Adventure* (-0.0027) received the strongest suppression, whereas *Non-gameplay/Tools* (-0.0004) and *Adult* (-0.0006) received the mildest penalties.

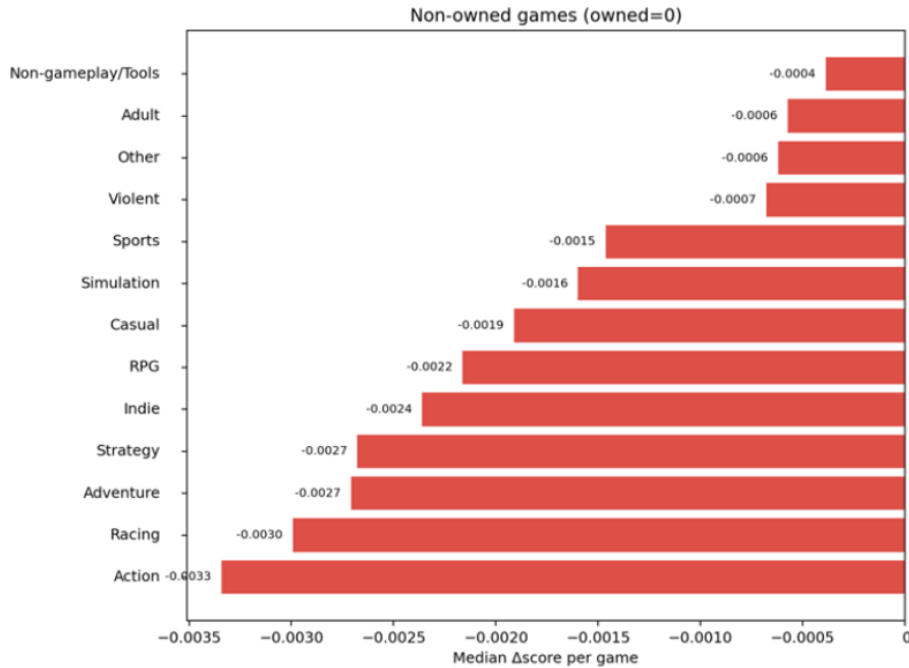


Figure 17: Median change in score per game for each genre for not-owned games

As shown in Figure 18, the ordering of genres closely mirrors the popularity ranking of those genres (how many users own them). There is a very strong *Spearman correlation* of -0.938 between the median score difference for not-owned games and the median user count per genre. This suggests that what really made an impact was not the genres themselves, but how popular the games were, which is consistent with the previous findings.

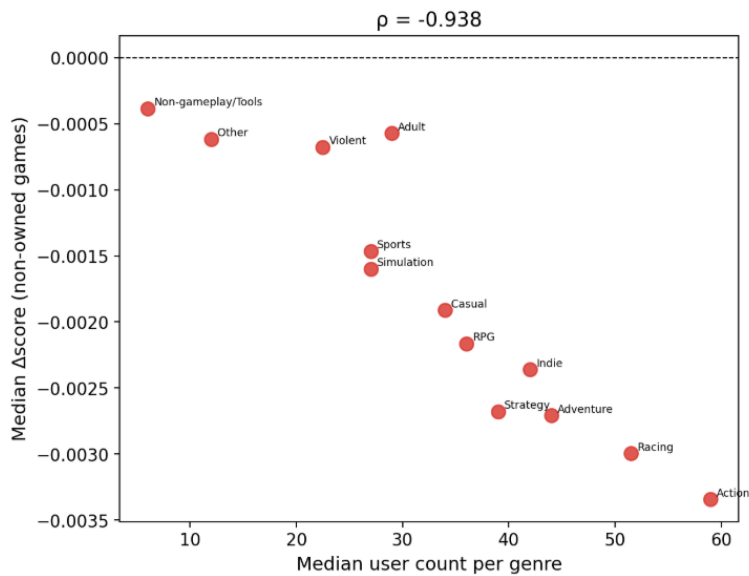


Figure 18: Correlation between median difference in score and median user count per genre

7.4.5 game_emb_0 and Popularity

The first embedding (*game_emb_0*) is highly correlated with the user count (*Spearman* = 0.977), as shown in *Figure 19*. The relationship follows a logarithmic-like curve: values of *game_emb_0* increase sharply for low-degree games and then flatten for very popular, high-degree games.

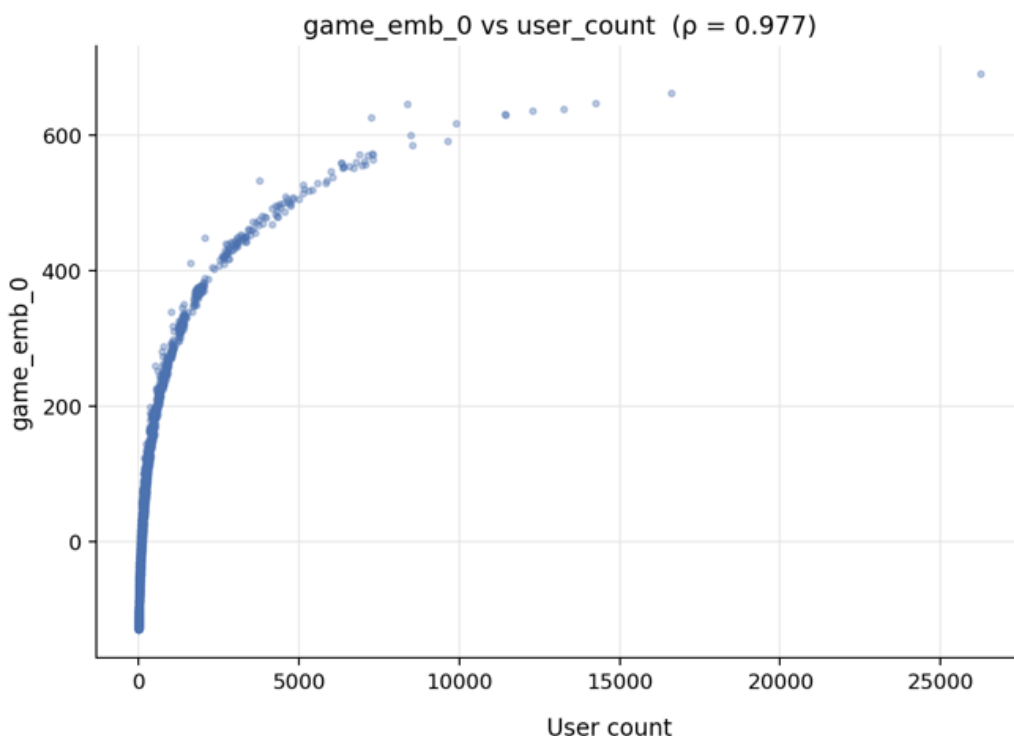


Figure 19: Correlation between game_emb_0 and user count

These findings made us think about whether *game_emb_0* might simply behave like a log-transformed *user_count* (since *game_emb_0* is so strongly correlated with popularity). Therefore, we ran the *XGBoost Baseline* with an additional $\log(\text{user_count})$ feature.

Additionally, we created a synthetic embedding that had the same correlation with the *user_count* but consisted of random numbers. This feature was generated directly from *user_count* by combining its standardized values with random noise, so that its *Spearman correlation* with *user_count* was approximately 0.97.

Both variants failed to reproduce the performance achieved by the real *game_emb_0*, and both *paired t-tests* showed no statistically significant differences compared with the original *XGBoost Baseline* model after *Holm correction* (see *Table 28*).

These results indicate that the network-derived features captured additional information that popularity measures could not provide.

Model	HitRate@1	HitRate@10	MRR
XGBoost Baseline with game_emb_0	0.9840 $\pm$ 0.0003	0.9936 $\pm$ 0.0001	0.9872 $\pm$ 0.0003
XGBoost Baseline	0.7768 $\pm$ 0.0004	0.8992 $\pm$ 0.0016	0.8187 $\pm$ 0.0006
XGBoost with log(user_count)	0.7730 $\pm$ 0.0039	0.8965 $\pm$ 0.0011	0.8150 $\pm$ 0.0034
XGB Baseline with synthetic game_emb_0	0.7749 $\pm$ 0.0019	0.8987 $\pm$ 0.0018	0.8172 $\pm$ 0.0015

Table 28: Comparison of XGBoost variants: Baseline with game_emb_0, Baseline, log(user_count), and synthetic game_emb_0

8 Discussion

These results suggest that network-derived features can significantly improve Steam game recommendation quality, but their benefit depends strongly on the model type. Traditional ML models showed the largest gains: *XGBoost* increased from less than 0.82 MRR in the *Baseline* variant to more than 0.99 in the *Network* variant. *Logistic Regression* improved by roughly 0.5 MRR (from ~ 0.34 to ~ 0.84), and *Random Forest* increased from below 0.7 to above 0.96.

Neural and *GCN* models did not reach competitive performance and did not improve statistically with the addition of network-derived features. For the *MLP*, this was likely related to the need to train in batches due to hardware and memory constraints, which limited model capacity. For *GraphSAGE*, an additional factor was that it already incorporates graph structure through its propagation layers, so the game embeddings may not have provided much additional signal on top of that. *LightGCN* performed better than both, indicating that the interaction graph alone was informative, although even *LightGCN* remained well below the strongest feature-based approaches.

PCA showed that game_emb_0 (the first component) explained more than 90% of the total variance, capturing most of the network signal, which was strongly influenced by popularity. *SHAP* analysis indicated that *XGBoost* relied primarily on game_emb_0 and user_count. These two features complemented each other: user_count captured raw global popularity, while game_emb_0 refined it by weighting ownership patterns through the social graph. Further experiments confirmed that using only these two features together yielded MRR ~ 0.96 , whereas individually they achieved only low MRR scores (~ 0.23 for game embeddings alone and ~ 0.27 for user_count alone).

Although the first embedding was almost perfectly correlated with *user_count* (Spearman 0.977), it was not just a log-transformed popularity measure. Adding $\log(\text{user_count})$ or a synthetic feature with the same correlation did not meaningfully change performance and did not differ significantly from the *Baseline*. This suggests that *game_emb_0* captured similarity information weighted by the social positions of game owners, providing a richer signal than popularity alone.

Network-derived features also altered the recommendations and score distribution. The first embedding suppressed popular games the user did not own and further boosted popular games the user did own, giving *XGBoost* a strong discriminative signal. This resulted in very high score separation (0.95 with *game_emb_0* vs. 0.59 in the *Baseline*). The Top-10 overlap was small, only 1.76 games on average, indicating that the model produced genuinely different and more accurate recommendations while remaining highly confident.

Finally, the model generalized well to unseen users, suggesting that it did not simply memorize user-specific patterns but learned meaningful relationships between users and games.

Overall, these patterns indicate that *Steam's* social network contains meaningful information about game similarity that non-network features alone do not capture. Despite the friendship graph's sparsity (density of 1.67×10^{-4}), it provided a strong and useful signal, and even a small number of network-derived features was sufficient to improve recommendation quality.

9 Limitations

Our research had several limitations related to data sampling, experimental setup, and hardware constraints that need to be acknowledged.

Sampling Bias and Popularity Bias

The data we sampled overrepresented users who chose to set their accounts as public (since all private profiles were excluded) and users who had at least one friend on Steam (as isolated users were never visited during the crawl). Because the crawl was initiated from our supervisor's *Steam ID*, the *randomized graph crawl* procedure was more likely to explore certain regions or tightly connected communities, potentially further overrepresenting specific user groups. Geographic imbalance was also observed, with users from China and the United States appearing disproportionately more often in the collected dataset (see *Figure 20*). Finally, the minimum ownership threshold of five games used during training excluded casual users, skewing the sample toward more active members of the platform.

The dataset was also biased toward popular games, which were more likely to be owned by the sampled users and therefore more frequently represented in the training data (see *Figure 21*),

increasing the model’s focus on those titles. Moreover, network-derived features were computed only for the games included in our sample, meaning that the model can operate exclusively on this subset. As a result, some niche titles and future releases are not supported by the model. To extend coverage beyond the sampled games, the embeddings would need to be recomputed to incorporate the full game catalog.

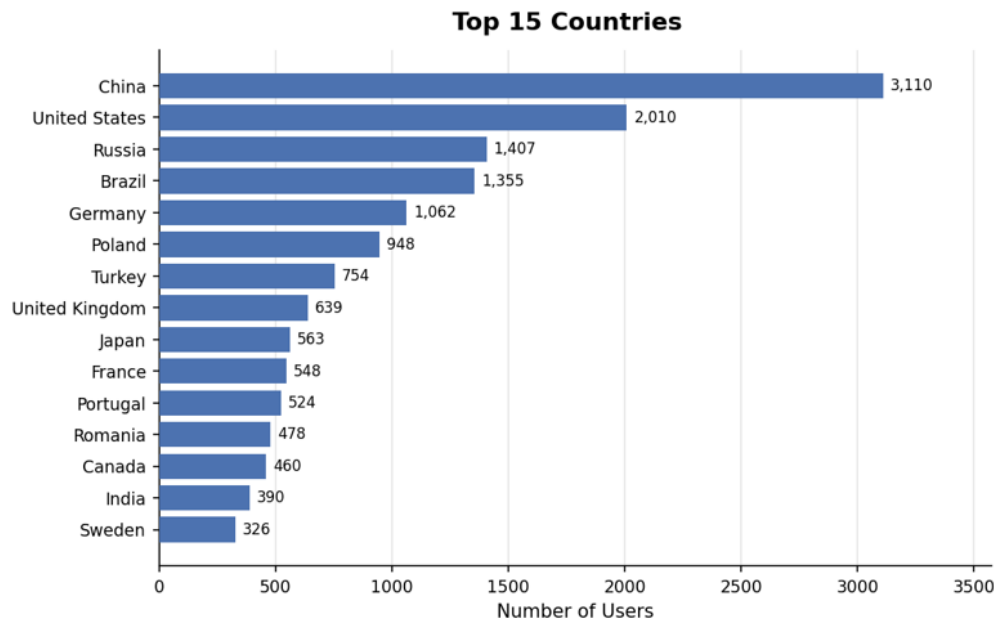


Figure 20: Top 15 countries with the most sampled users

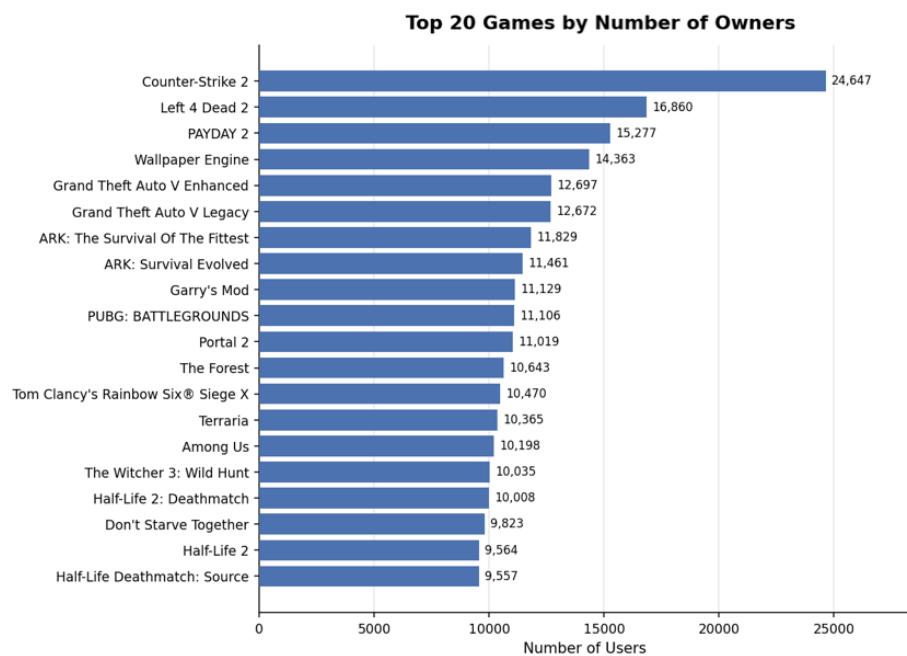


Figure 21: Top 20 games by number of owners

Simplification in Recommender Design

Our problem formulation also simplified the real recommendation task. The model was required to recommend only one game per user, and it selected from a set of just *101* candidate games, which represented only a small fraction of Steam’s full catalog and therefore made the task easier. Unfortunately, reducing potential candidates was necessary to keep computation feasible on such a large dataset.

Moreover, the positive game for each user in the validation and test sets was sampled randomly from the user’s library due to the lack of temporal information about when games were acquired. As a result, we could not model the evolution of user preferences over time, and the model effectively learned to predict ownership, not what a user would want to play next.

Furthermore, owning a game does not necessarily imply that the user likes or actively engages with it. In fact, 44.9% of all user-game pairs had zero playtime, meaning that many users purchase games (or receive them as gifts) without ever playing them. The *CCDF* of playtime further shows that most users play only briefly, with a median of just 4.5 hours per user-game pair (see *Figure 22* and *Table 29*). Such low playtime values may suggest that the user did not enjoy the game. A proper recommender system should be able to distinguish between ownership and genuine interest.

The lack of important behavioral signals, such as clicks, browsing sessions, or short-term interaction patterns, further limits our approach. These signals are commonly used in industrial recommender systems and could improve performance by providing more information about users’ intent.

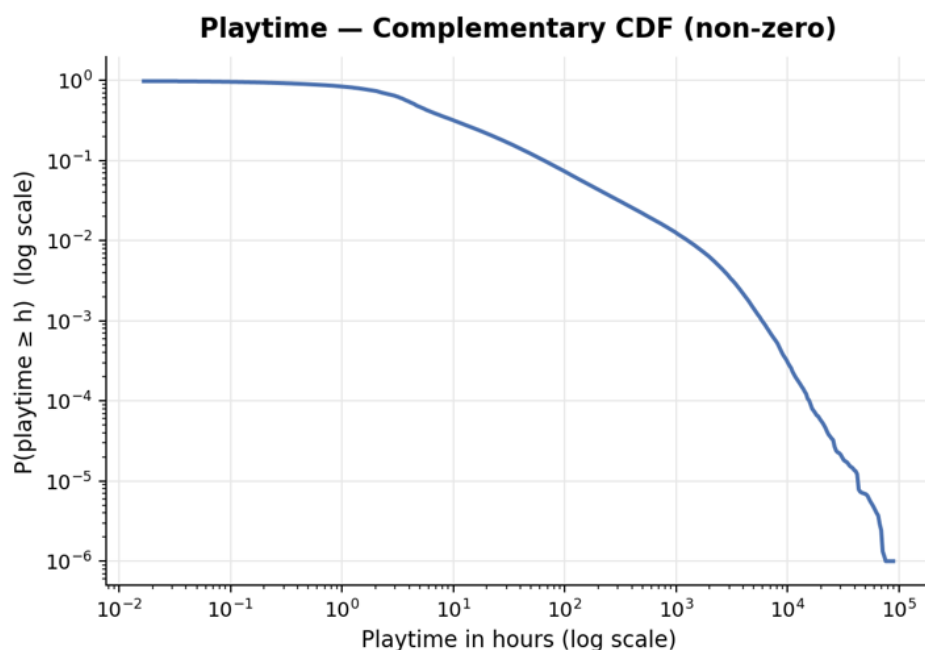


Figure 22: CCDF of playtime for non-zero values

Metric	Value in hours
median	4.5h
mean	63.5h
max	88,985h

Table 29: Summary statistics of playtime per user-game pair

Hardware Limitations

Our datasets were very large: the training set contained 43,494,595 rows (26 GB), while both the validation and test sets contained 2,849,312 rows each (1.7 GB). Because we prioritized keeping as much useful data as possible, we had to optimize the training process carefully. Training the models, particularly the *Neural Network* and *GraphSAGE*, required enormous amounts of RAM, which exceeded the capacity of the university’s HPC cluster. As a result, many training runs required batching, careful memory management, and strict limits on model complexity. These constraints restricted our ability to explore larger feature spaces, deeper neural architectures, and broader hyperparameter searches.

10 Conclusion

In this thesis, we investigated whether network-derived features can improve machine learning models for *Steam* game recommendations. Using a large-scale dataset of user-game interactions, friendship relations, user attributes, and game attributes, we evaluated multiple recommendation approaches under both *Baseline* and *Network-enhanced* settings.

Across traditional machine learning models, incorporating network-derived features led to significant improvements in recommendation quality, with *XGBoost* achieving the strongest overall performance. Neural and graph-based models did not benefit meaningfully from the additional features, likely due to hardware constraints and architectural differences. The gains in traditional models were driven primarily by the interaction between the popularity feature *user_count* and the first embedding component *game_emb_0*.

The embedding captured structural relationships within the social ownership graph, offering information that could not be reproduced by simple popularity measures. This resulted in more accurate rankings and clearer score separation between relevant and irrelevant games. The models also generalized well to unseen users, indicating that they learned meaningful patterns rather than memorizing user-specific behavior.

Overall, the findings show that even highly sparse social networks contain valuable structural information that can significantly enhance recommendation systems when transformed into suitable feature representations. However, because our evaluation used ownership rather than future engagement, the results should not be interpreted as evidence for deployment ready recommendation quality.

Future work could incorporate temporal dynamics, stricter evaluation setups, and more representative datasets. It would also be valuable to test whether similar network-derived embeddings improve recommendations on other platforms, where user-item interactions follow different patterns.

References

- Coscia, Michele. “Generalized Euclidean measure to estimate network distances.” In Proceedings of the international AAAI conference on web and social media, vol. 14, pp. 119-129. 2020. <https://ojs.aaai.org/index.php/ICWSM/article/view/7284/7138>
- Coscia, Michele. “Node attribute analysis for cultural data analytics: a case study on Italian XX–XXI century music.” Applied Network Science 9, no. 1. 2024: 56.
- Damstrup, Anne Sophie Riis, Sofie Tosti Madsen, and Michele Coscia. “Unsupervised Learning via Network-Aware Embeddings”, arXiv preprint arXiv:2309.10408. 2023. <https://arxiv.org/abs/2309.10408>
- Hamilton, William L., Rex Ying, and Jure Leskovec. “Inductive representation learning on large graphs.” In Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS), pp. 1025–1035. 2017. arXiv. <https://doi.org/10.48550/arxiv.1706.02216>
- He, Xiangnan, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. “LightGCN: Simplifying and powering graph convolution network for recommendation.” In Proceedings of the 43rd international ACM SIGIR conference on research and development in information retrieval, pp. 639-648. 2020. arXiv. <https://doi.org/10.48550/arxiv.2002.02126>
- Holm, Sture. “A simple sequentially rejective multiple test procedure”, Scandinavian Journal of Statistics 6, no. 2 (1979): 65–70. <https://www.jstor.org/stable/4615733>
- Koren, Yehuda, Robert Bell, and Chris Volinsky. “Matrix factorization techniques for recommender systems.” Computer 42, no. 8 (2009): 30–37 [https://datajobs.com/data-science-repo/Recommender-Systems-\[Netflix\].pdf](https://datajobs.com/data-science-repo/Recommender-Systems-[Netflix].pdf)
- Lundberg, Scott M., and Su-In Lee. “A unified approach to interpreting model predictions.” In Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS), pp. 4768-4777. 2017. arXiv. <https://doi.org/10.48550/arXiv.1705.07874>
- Rendle, Steffen, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. “BPR: Bayesian Personalized Ranking from Implicit Feedback.” In Proceedings of the 25th Conference on Uncertainty in Artificial Intelligence (UAI), pp. 452–461. 2009. arXiv. <https://doi.org/10.48550/arXiv.1205.2618>
- Ricci, Francesco, Lior Rokach, Bracha Shapira, and Paul B. Kantor. Recommender Systems Handbook. Boston, MA: Springer, 2011. https://www.researchgate.net/publication/227268858_Recommender_Systems_Handbook
- Sinaga, V. L., and A. Wibowo. “Combining XGBoost and hybrid filtering algorithm in e-commerce recommendation system.” 2024.

https://www.researchgate.net/publication/392569863_Combining_XGBoost_and_hybrid_filtering_algorithm_in_e-commerce_recommendation_system#fullTextFileContent

Valve Corporation. “Introducing the Steam Interactive Recommender.” Steam News, 2020.
<https://store.steampowered.com/news/app/593110/view/1716373422378712840>

Valve Corporation. “Steam Friends List.” Steam Support, 2026.
<https://help.steampowered.com/en/faqs/view/1902-2D54-E445-59DD>

Valve Corporation. “Steam Profile Privacy.” Steam Support, 2026.
<https://help.steampowered.com/en/faqs/view/588C-C67D-0251-C276>

Valve Corporation. “Steam Web API Documentation.” Steam Community, 2026.
<https://steamcommunity.com/dev>

Appendix

The repository contains all code used for data preprocessing, model training, and evaluation, as well as the scripts required to reproduce the experiments reported in this thesis:

https://github.com/kubaa1501/bachelor_project

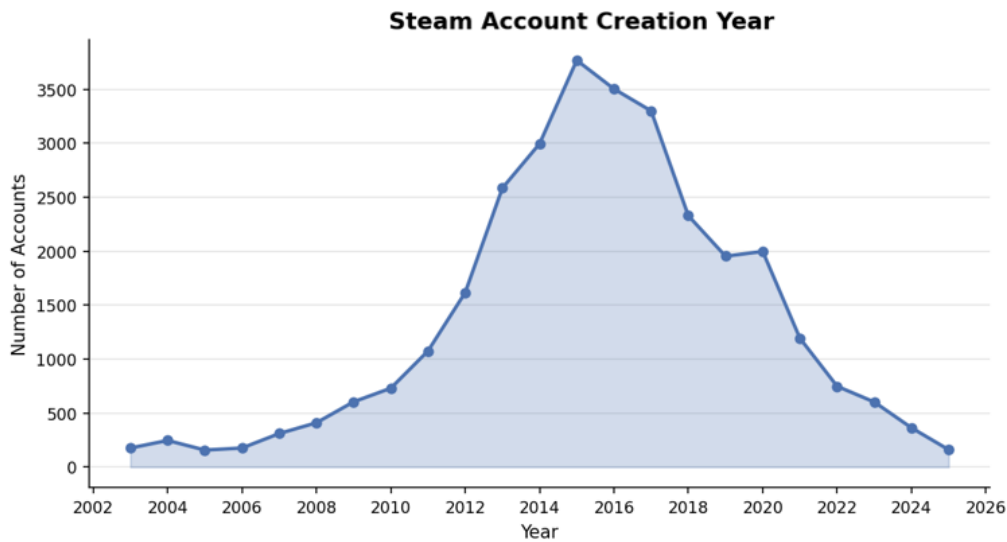


Figure 1: Number of accounts created each year

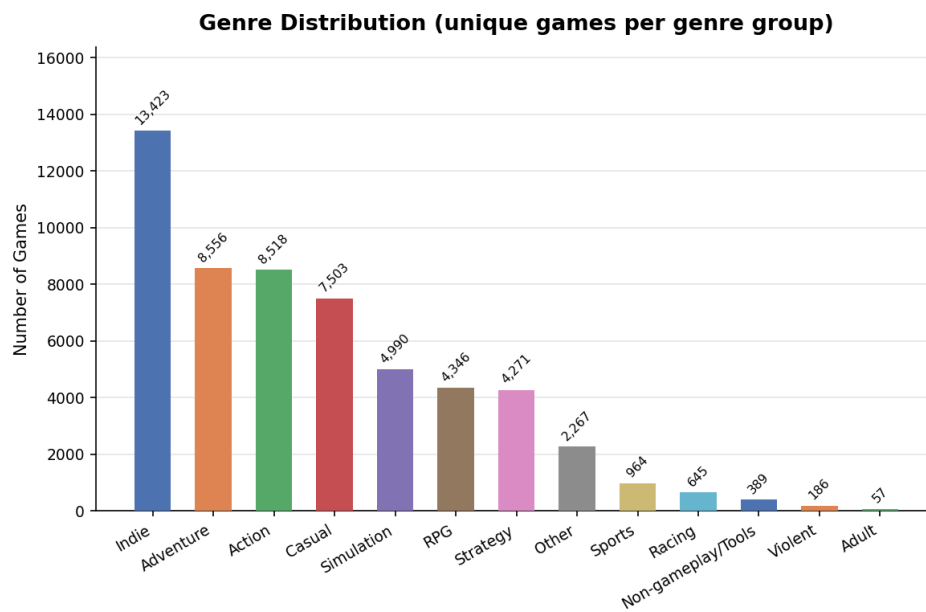


Figure 2: Number of unique games per genre group

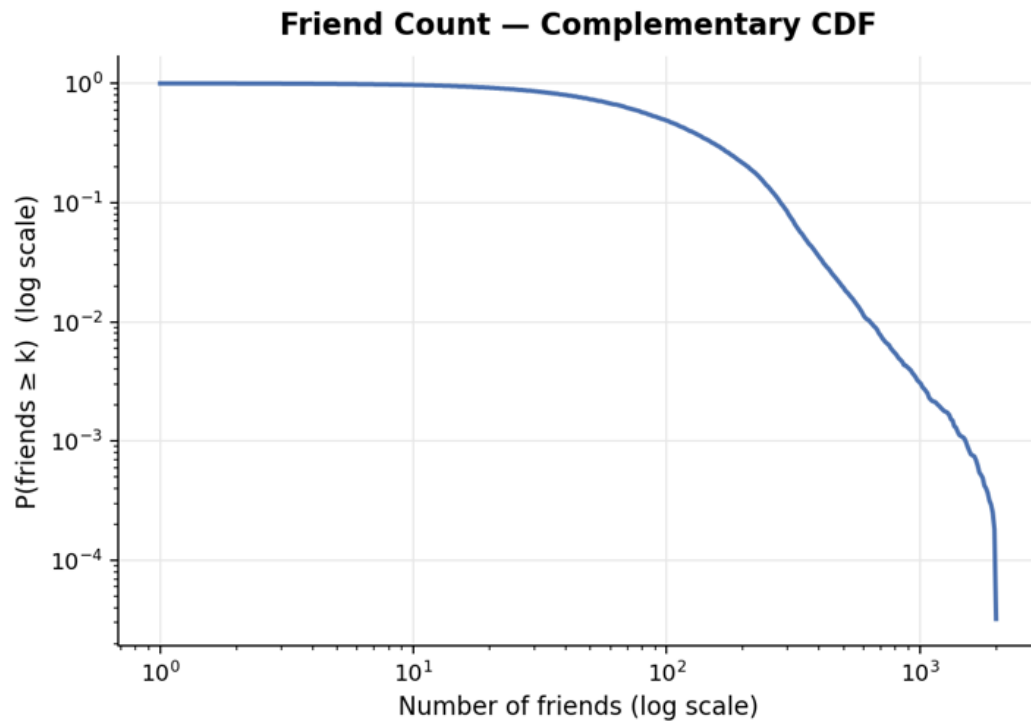


Figure 3: CCDF of the number of friends per user

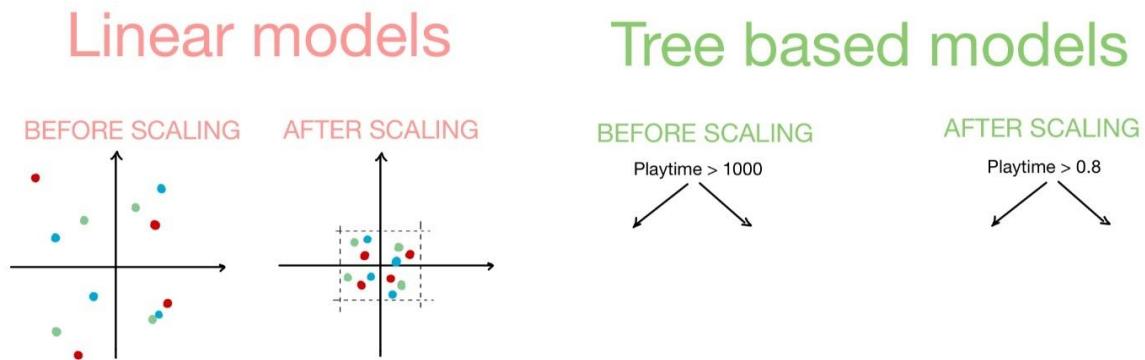


Figure 4: Effect of feature scaling on different model types.

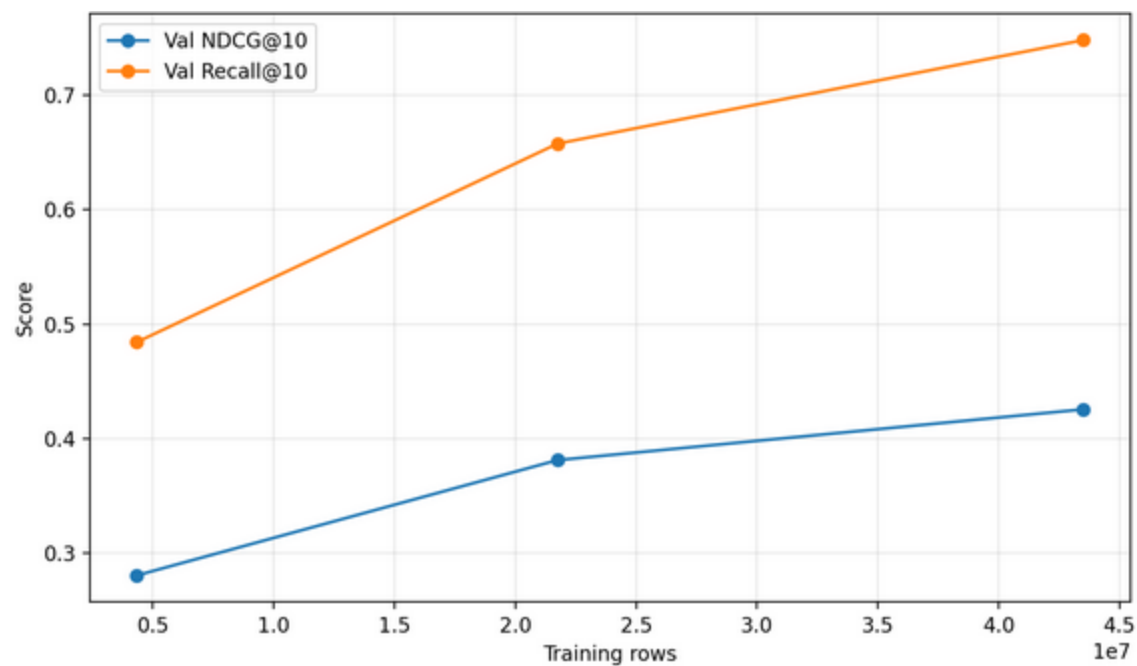


Figure 5: Ranking learning curve for Logistic Regression Baseline model (multiseed version)

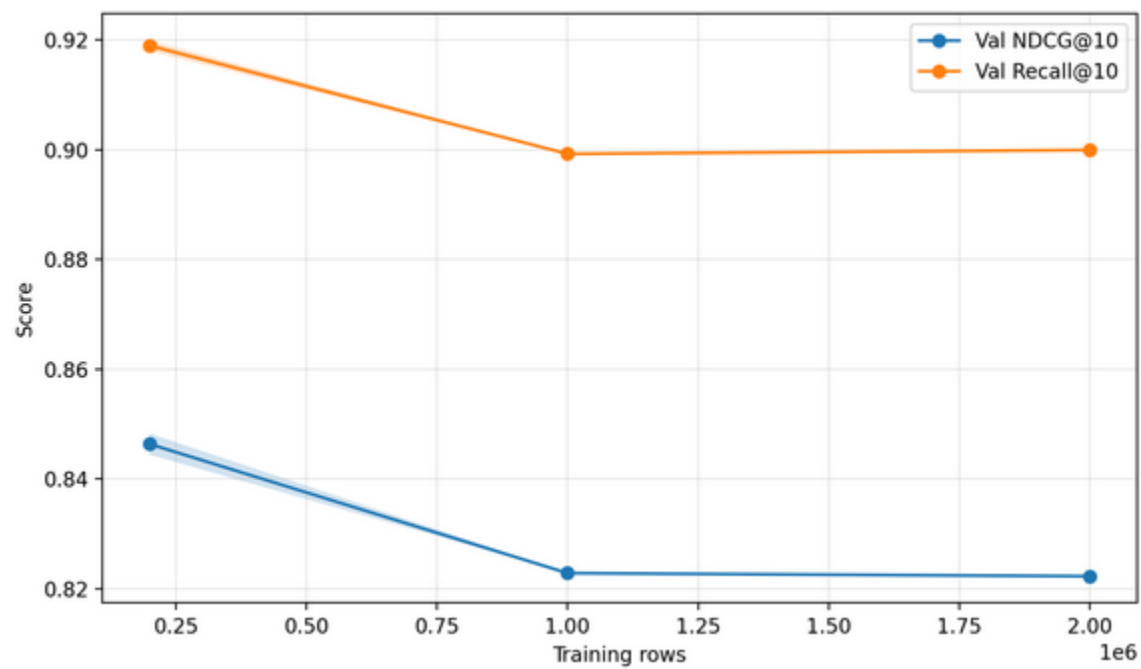


Figure 6: Ranking learning curve for Logistic Regression Network model (multiseed version)

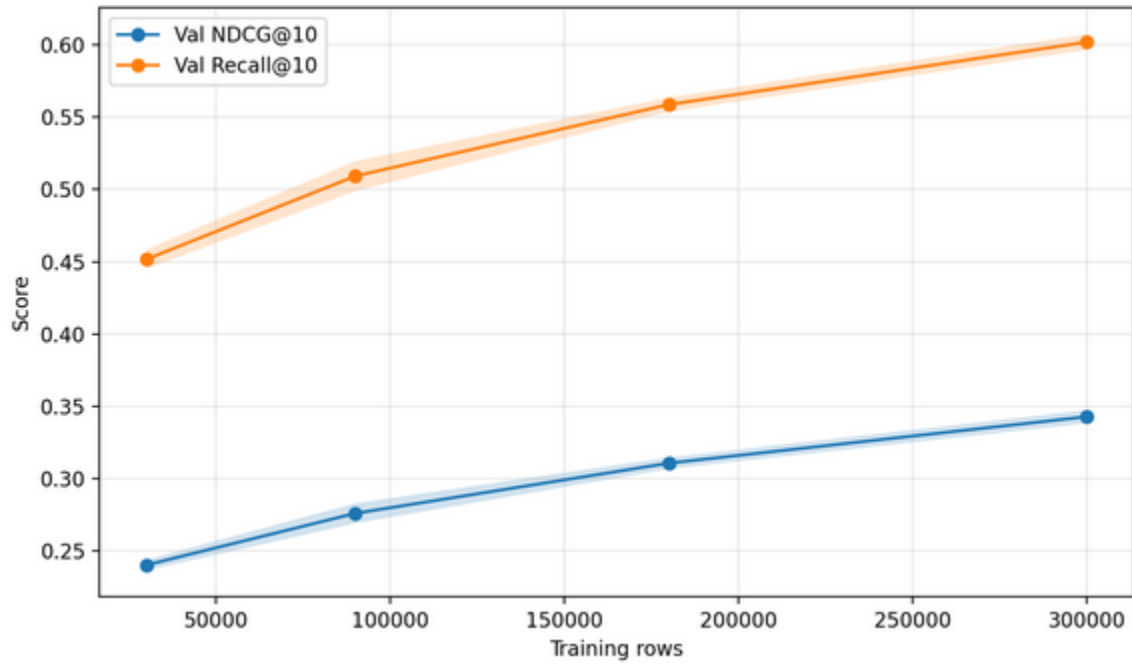


Figure 7: Ranking learning curve for Random Forest Baseline model (multiseed version)

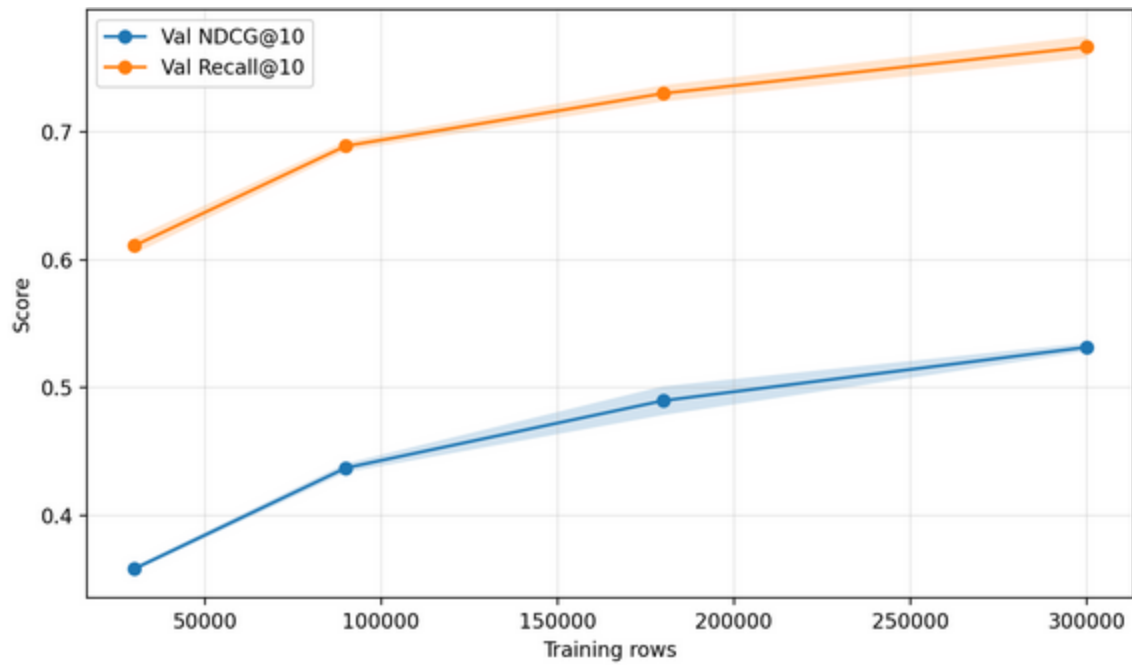


Figure 8: Ranking learning curve for Random Forest Network model (multiseed version)

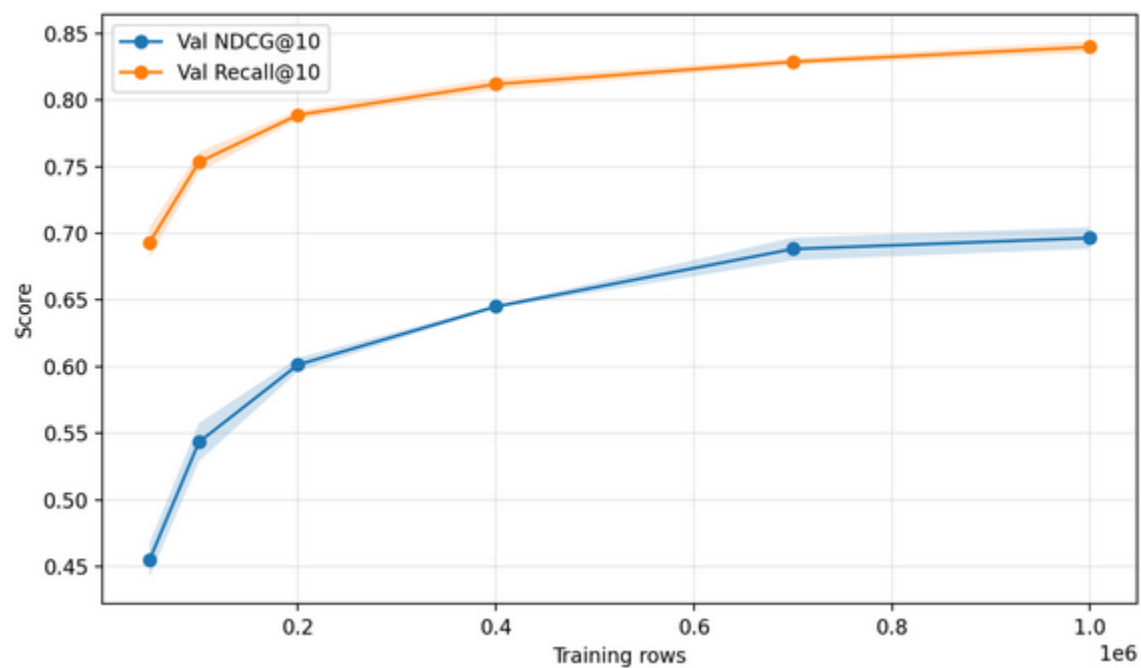


Figure 9: Ranking learning curve for XGBoost Baseline model (multiseed version)

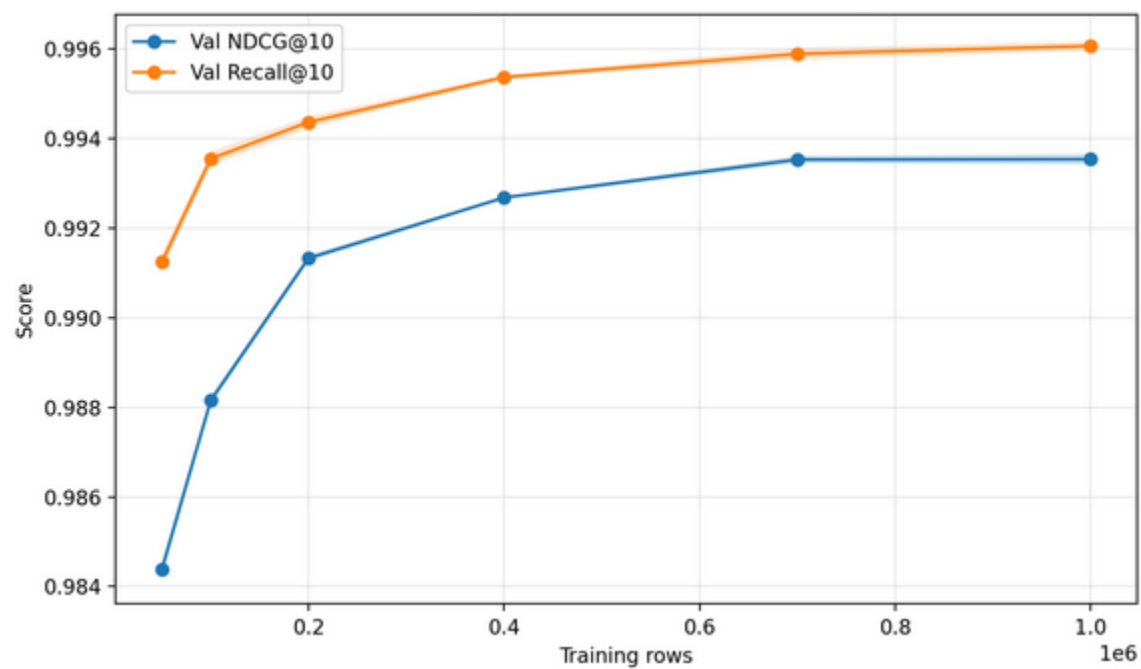


Figure 10: Ranking learning curve for XGBoost Network model (multiseed version)

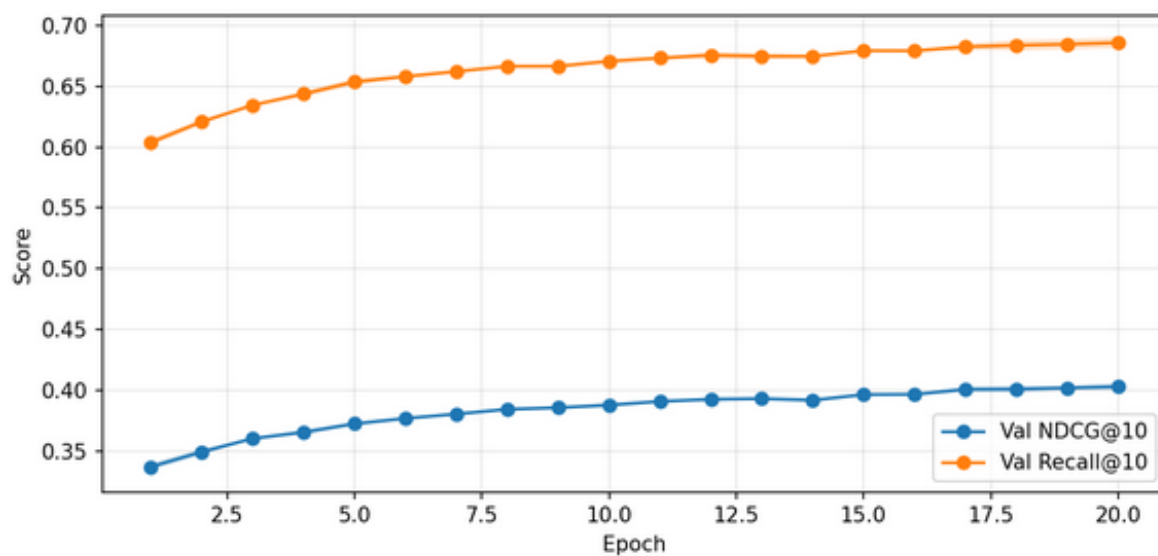


Figure 11: Ranking learning curve for Neural Network Baseline model (multiseed version)

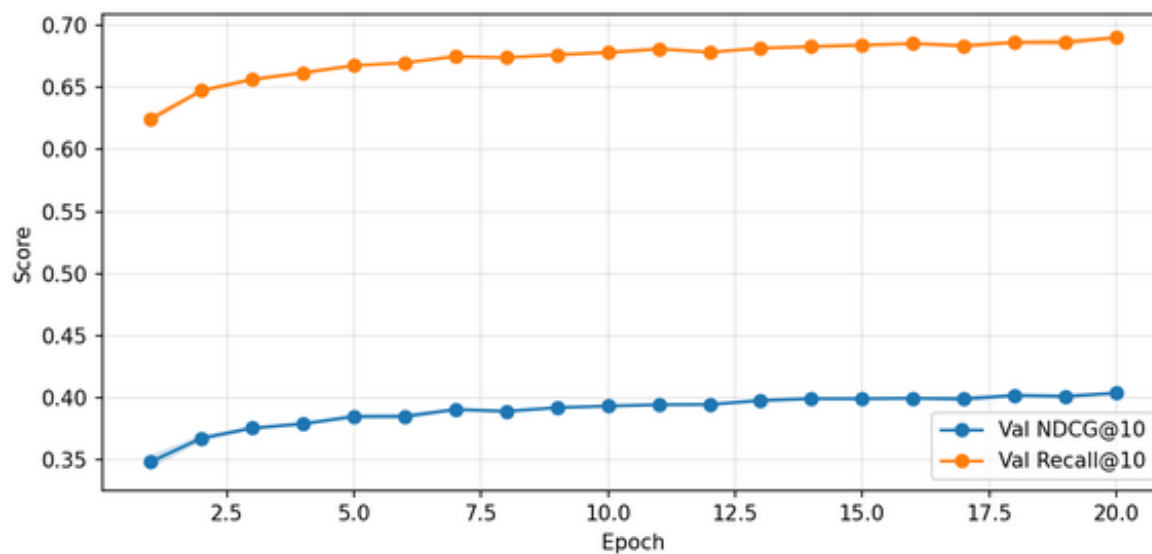


Figure 12: Ranking learning curve for Neural Network Network model (multiseed version)

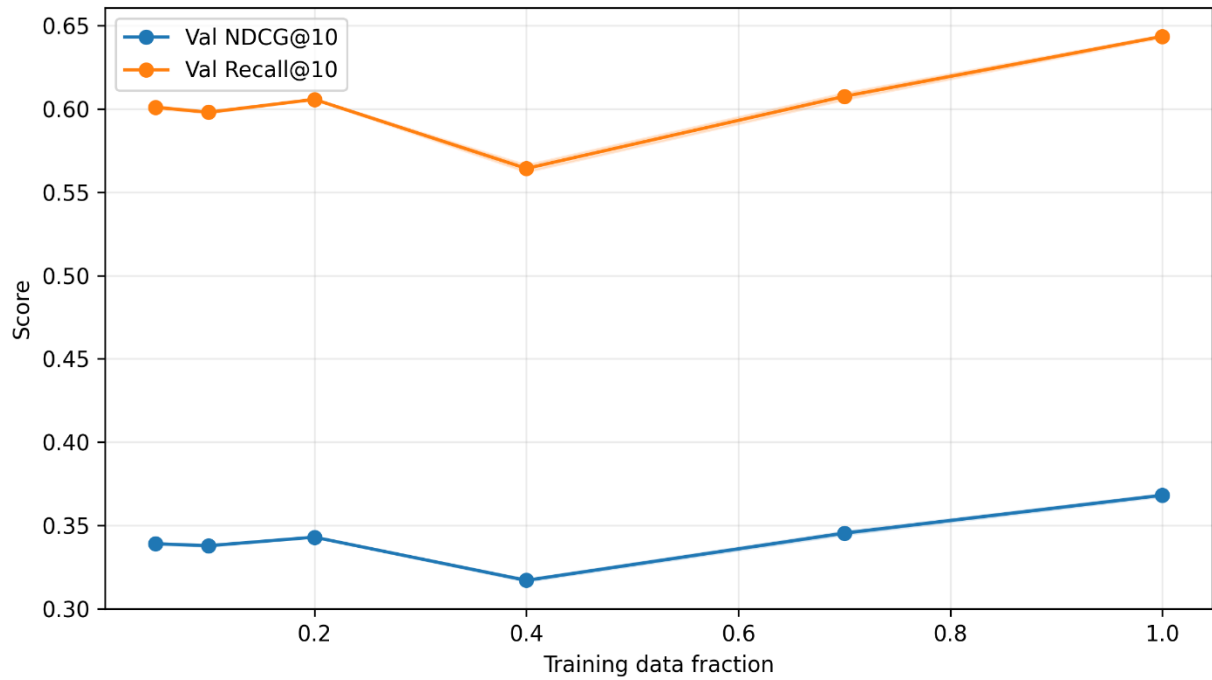


Figure 13: Ranking learning curve for LightGCN model (multiseed version)

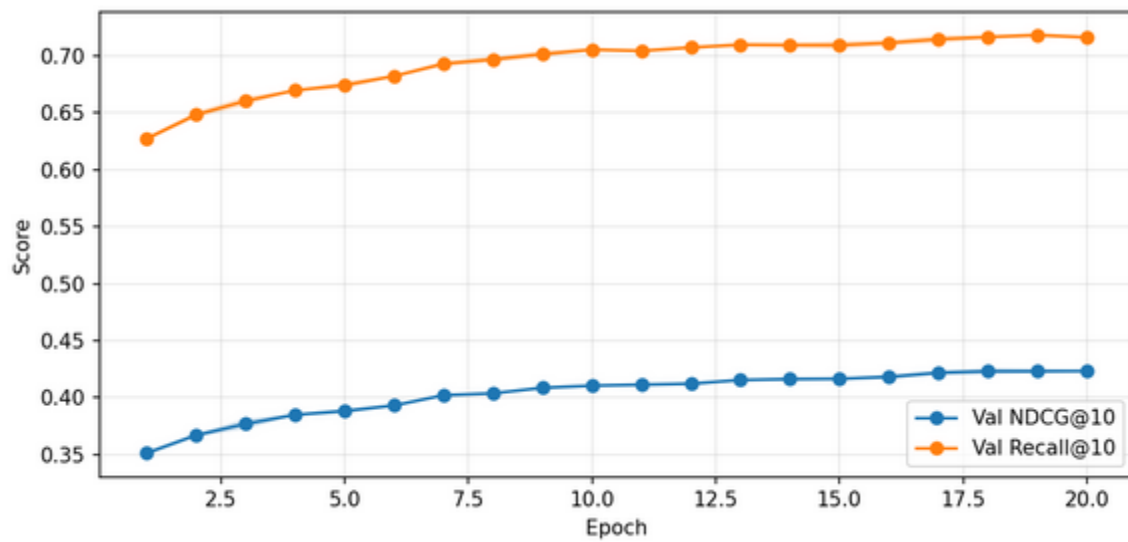


Figure 14: Ranking learning curve for GraphSAGE Baseline model (multiseed version)

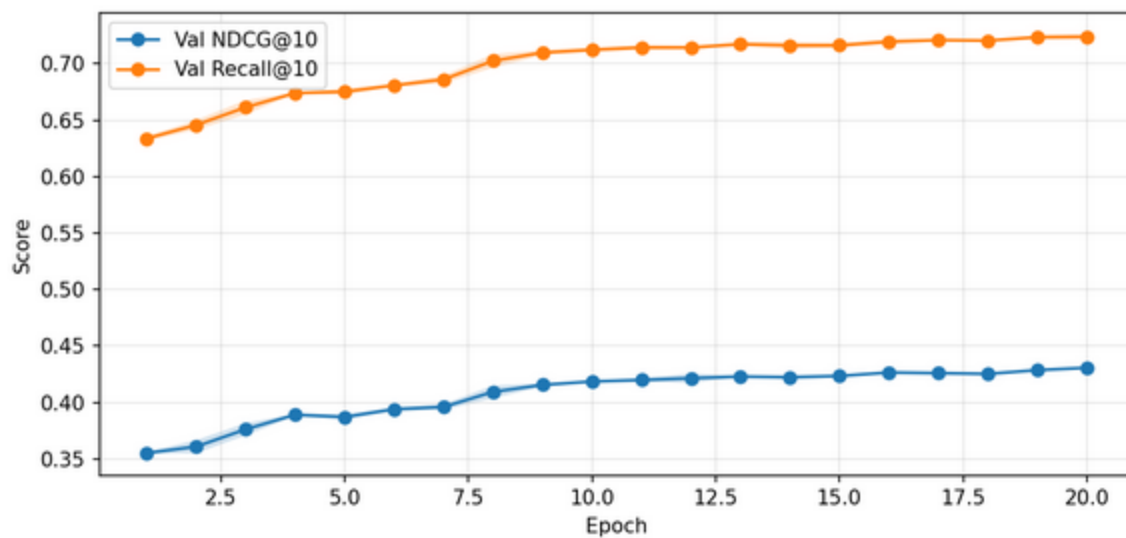


Figure 15: Ranking learning curve for GraphSAGE Network model (multiseed version)

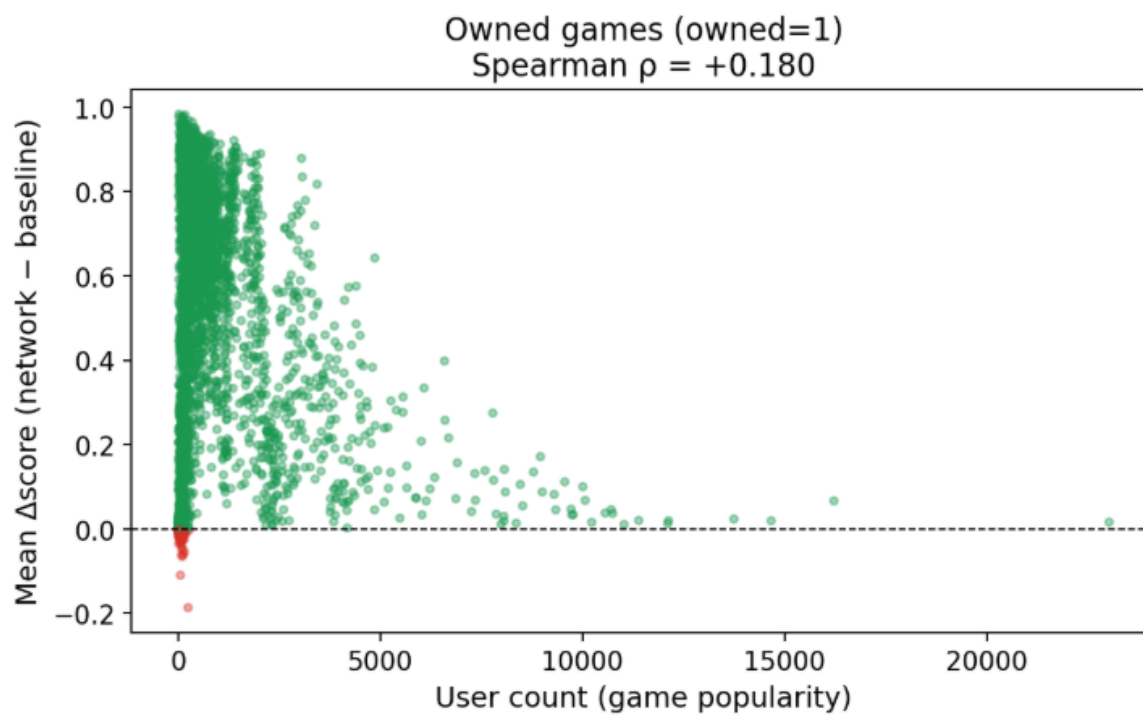


Figure 16: Score change vs game popularity for actually owned games

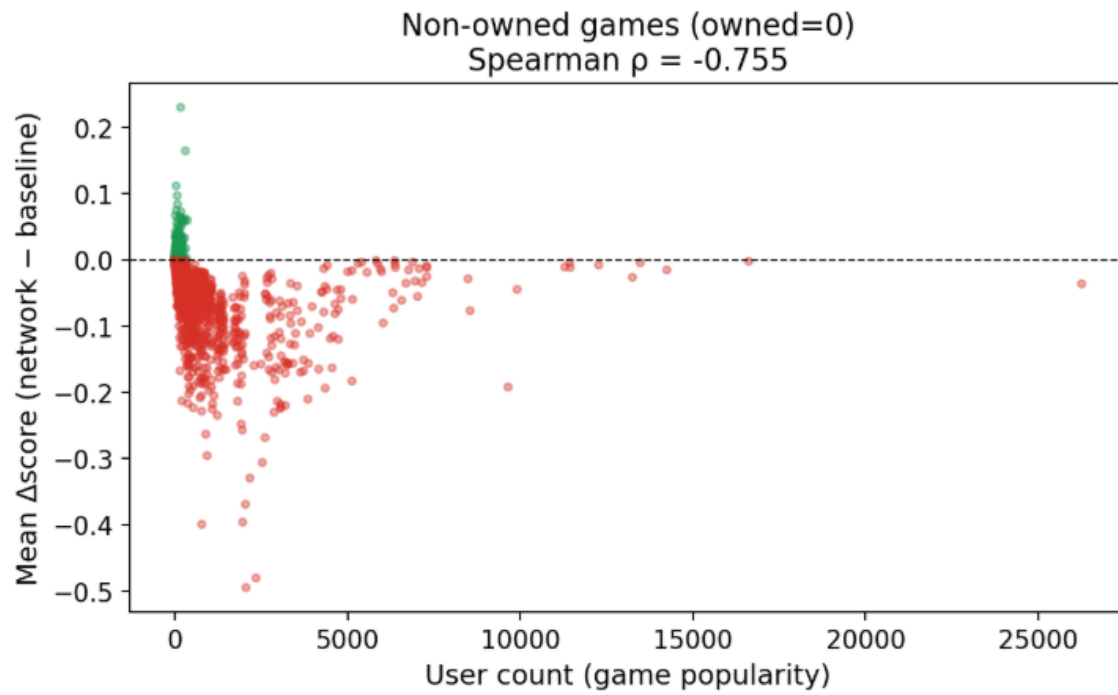


Figure 17: Score change vs game popularity for not-owned games